

Edición 2026 · Niveles 1 a 7

DATOS

Guardar muchos: listas, tuplas y diccionarios

Ya sabes guardar un dato, decidir con él y repetirlo mil veces. Pero tus programas siguen teniendo un techo: **una variable, un dato**. Este es el tomo donde ese techo desaparece. Aquí tu código deja de manejar números sueltos y empieza a manejar **colecciones**: playlists, inventarios, agendas, notas de un curso entero.

-
- | | |
|--|--|
| 1 Listas · la caja con casillas numeradas | 2 Listas vivas · crecer, encoger y ordenar |
| 3 Listas + ciclos · donde todo encaja | 4 Tuplas · lo que no se debe tocar |
| 5 Diccionarios · buscar por nombre, no por número | 6 Anidar · el mundo real es una lista de fichas |
| 7 Depurar y proyecto · el jefe final | A Anexos · chuletas, errores y repaso |
-

POR QUÉ EN ORDEN

Cada nivel usa lo del anterior como material de construcción. Saltarte uno no te ahorra tiempo: te lo cobra después, con intereses.

Cómo está construido este tomo

Léelo una vez. Son tres minutos que te ahorran siete niveles de confusión.

01 El mapa de los siete niveles

Este tomo no está escrito para que lo leas: está escrito para que lo **ejecutes**. Los tres primeros niveles te dan la estructura más usada del mundo y la conectan con los ciclos que ya sabes. El cuarto y el quinto te dan las otras dos formas de guardar datos. Los dos últimos te enseñan a combinarlas y a arreglar lo que se rompa.

NIVEL	TEMA	QUÉ SABRÁS HACER AL SALIR	ESTADO
1	Listas	Guardar muchos datos en una sola caja y sacar cualquiera por su posición	Empieza aquí
2	Listas vivas	Agregar, borrar, buscar y ordenar sin romper nada	Núcleo
3	Listas + ciclos	Recorrer colecciones y aplicar los cuatro patrones del tomo 2	Núcleo
4	Tuplas	Proteger datos que no deben cambiar y devolver varios valores a la vez	Precisión
5	Diccionarios	Buscar por nombre en vez de por número y contar cualquier cosa	Salto grande
6	Anidar	Manejar tablas, matrices y fichas: la forma real de los datos	Aplicación
7	Depurar + proyecto	Cazar los bugs típicos de colecciones y entregar un programa completo	Jefe final

POR QUÉ COLORES

Cuando una información llega por dos canales a la vez (palabra + color), se recuerda mejor que por uno solo. Aquí el color siempre significa lo mismo: es una etiqueta, no un adorno.

02 El código de color: aprende a leer sin leer

Cada tipo de contenido tiene un color fijo. Después de dos niveles vas a reconocer un recuadro antes de leer su título, y eso te deja decidir en un segundo si lo lees ahora o lo dejas para el repaso.

ÍNDIGO • CONCEPTO CLAVE

Definiciones y estructura. Es lo que tendrías que subrayar si esto fuera papel.

VERDE • TIP Y BUENA PRÁCTICA

El atajo que usan los que ya llevan tiempo. Poco esfuerzo, mucho resultado.

ÁMBAR • ERROR CLÁSICO

La trampa en la que cae todo el mundo. Leerlo antes te ahorra media hora de pelea.

ROJO • PELIGRO REAL

Cosas que rompen tu programa o lo hacen mentir en silencio. Poco frecuentes, muy serias.

MAGENTA • TRAZA PASO A PASO

La simulación a mano, vuelta por vuelta. La herramienta central del tomo anterior, ahora con cajas.

VIOLETA • ¿SABÍAS QUÉ?

Historia y cultura de la programación. Opcional para el examen, obligatorio para la conversación.

TURQUESA • EN LA VIDA REAL

Dónde está esto funcionando ahora mismo en apps que usas todos los días.

BLANCO CON BORDE • LABORATORIO

La práctica guiada de la sesión. Se hace con el editor abierto, no leyendo.

03 Las cuatro reglas de la casa

1 • ESCRÍBELO, NO LO COPIES

Teclear cada ejemplo a mano instala la sintaxis en tus dedos. Copiar y pegar instala la ilusión de haber entendido.

2 • PREDICE ANTES DE EJECUTAR

Antes de darle a ejecutar, escribe qué crees que va a salir. Acertar confirma; fallar enseña el doble.

3 • RÓMPELO A PROPÓSITO

Cambia un índice, borra una línea, mete un dato absurdo. Una estructura que se rompe frente a ti enseña más que diez que funcionan.

4 • EXPLÍCALO SIN MIRAR

El nivel es tuyo cuando puedes contarlo en voz alta con tus palabras. Si tienes que volver a la página, todavía no.

ANTES DE PASAR LA PÁGINA

Abre tu editor y déjalo en una ventana al lado. Y abre también la **consola interactiva** (escribe `python` en la terminal): en este tomo vas a querer probar cosas sueltas todo el tiempo, y ahí se prueban en dos segundos.

PRÁCTICA ESPACIADA

Estudiar 4 días de 45 minutos deja más huella que un día de 3 horas. El olvido parcial entre sesiones es justamente lo que fortalece el recuerdo.

04 Ruta de estudio: cuatro sesiones, no una tarde

Puedes leer los siete niveles en una tarde. No lo hagas. Las estructuras de datos son el primer tema donde el problema no es entender una línea, sino **imaginar la forma que tienen tus datos en la memoria**. Eso necesita reposo entre sesiones.

SESIÓN	CONTENIDO NUEVO	REPASO OBLIGATORIO ANTES DE EMPEZAR
Día 1 · 50 min	Niveles 1 y 2	Del tomo 2: <code>range()</code> y por qué el fin se excluye
Día 2 · 50 min	Nivel 3 + laboratorio	Los cuatro patrones: contador, acumulador, campeón y bandera
Día 3 · 50 min	Niveles 4 y 5	Explica en voz alta qué hace <code>for letra in palabra</code>
Día 4 · 70 min	Niveles 6 y 7 + proyecto	Las tarjetas de repaso del final, tapando la respuesta
Día 12 · 25 min	—	Vuelve a hacer el laboratorio que peor te salió. Ese es el repaso que cuenta

05 Chequeo previo: ¿estás listo?

Todo este tomo se apoya en cinco cosas de los tomos anteriores. Respóndelas de memoria. Si fallas dos o más, vuelve una hora atrás: no es un retroceso, es afilar el hacha.

LO QUE YA DEBES TRAER PUESTO

- ¿Qué valores exactos produce `range(1, 6)` ?
- ¿Qué devuelve siempre `input()` , escriba lo que escriba la persona?
- ¿Dónde se inicializa un acumulador: dentro o fuera del bucle? ¿Por qué?
- ¿Qué hace `break` y en qué se diferencia de `continue` ?
- Escribe una f-string que muestre `promedio` con dos decimales.

RESPUESTAS EN UNA LÍNEA

1, 2, 3, 4, 5 · un `str` · fuera, porque dentro se reinicia cada vuelta · corta el bucle entero frente a saltar solo esta vuelta · `f"{promedio:.2f}"`

LA IDEA QUE ORDENA TODO EL TOMO

Hasta hoy, cada dato necesitaba su propia variable con su propio nombre. A partir de aquí existe otra opción: **una sola variable que contiene muchos datos**. A eso se le llama una **colección o estructura de datos**.

Python trae tres de bolsillo, y elegir bien entre ellas es la mitad de programar bien: la **lista** (ordenada y modificable), la **tupla** (ordenada y bloqueada) y el **diccionario** (buscado por nombre). Ese es todo el mapa.

Listas: una caja con casillas numeradas

La estructura más usada de Python. La que convierte veinte variables sueltas en una sola cosa con la que se puede trabajar.

LO QUE DESBLOQUEARÁS

- 1 Crear una lista y explicar por qué es distinta de una variable normal.
- 2 Sacar cualquier elemento por su **índice**, contando desde cero y desde el final.
- 3 Medir una lista con `len()` y evitar el error de índice.
- 4 Cortar trozos de una lista con **rebanadas** sin copiar a mano.

PALABRAS CLAVE

lista [] elemento
índice indexar len()
IndexError
índice negativo
rebanada slicing
colección

DEL TOMO 1

Una variable es una caja con un nombre y un valor. Una lista es una caja con un nombre y muchos valores numerados por dentro.

1.1 El problema que ninguna variable suelta resuelve

Imagina que tienes que guardar las notas de cinco materias. Con lo que sabes hasta hoy, la única salida es esta:

```
a_la_fuerza.py
1 nota1 = 4.5
2 nota2 = 2.8
3 nota3 = 3.9
4 nota4 = 5.0
5 nota5 = 3.0
6
7 promedio = (nota1 + nota2 + nota3 + nota4 + nota5) / 5
```

Funciona. Y es una mala solución, por la misma razón que lo eran los cinco `print` del tomo 2: **si mañana son cuarenta notas, tu programa no crece — se derrumba**. Peor todavía: **no puedes recorrer variables sueltas con un ciclo**. No existe forma de escribir «para cada notaN...», porque los nombres los inventaste tú uno por uno.

Esta es la misma información guardada en una **lista**. Léela despacio: cabe en una línea y funcionaría igual con cinco notas que con cinco mil.

● con_lista.py

```
1 notas = [4.5, 2.8, 3.9, 5.0, 3.0]
2
3 print(notas)           # toda la caja
4 print(notas[0])        # solo la primera casilla
5 print(len(notas))      # cuántas casillas hay
```

SALIDA EN PANTALLA

```
[4.5, 2.8, 3.9, 5.0, 3.0]
4.5
5
```

CONCEPTO CLAVE

Lista: una colección *ordenada* y *modificable* de valores, guardada en una sola variable. Se escribe entre **corchetes** `[ ]` y sus valores se separan con comas.

Elemento: cada valor que vive dentro de la lista.

Índice: el número de la casilla donde vive cada elemento. Es la dirección del elemento dentro de la caja, y empieza en 0.

TIP BÁSICO

Nombra las listas en **plural**:
`notas`, `precios`,
`alumnos`. Así, cuando
después escribas `for`
`nota in notas`, la
línea se lee sola.

1.2 Cómo se crea una lista

Hay tres formas de nacer, y las vas a usar las tres:

● crear_listas.py

```
1 materias = ["Matemáticas", "Español", "Física"] # 1. con valores
2 pendientes = [] # 2. vacía, para llenarla luego
3 letras = list("PYTHON") # 3. a partir de otra cosa
4
5 mezclada = ["Ana", 16, 4.5, True] # se permite mezclar tipos
6
7 print(letras)
8 print(pendientes, len(pendientes))
```

SALIDA EN PANTALLA

```
['P', 'Y', 'T', 'H', 'O', 'N']
[] 0
```

Que *se permita* mezclar tipos no significa que convenga. Una lista es mucho más útil cuando todos sus elementos son **la misma clase de cosa**: todas notas, todos nombres, todos precios.

Si necesitas guardar cosas distintas de una misma persona (nombre, edad, nota), lo que buscas es un **diccionario**, y llega en el nivel 5.

BUENA PRÁCTICA • LA LISTA VACÍA ES UN PUNTO DE PARTIDA LEGÍTIMO

`pendientes = []` no es una lista «mal hecha»: es exactamente el equivalente de `suma = 0` del tomo 2. Es un **acumulador de cosas**, preparado antes del bucle para llenarse dentro. Lo usarás en el nivel 3 constantemente.

REGLA DE ORO

La primera casilla es la **0** y la última es **len - 1**. Escríbelo en tu cuaderno hoy y ahórrate diez errores esta semana.

1.3 El índice: la dirección de cada casilla

Una lista no es un montón: es una **fila ordenada**. Cada elemento tiene un número de posición fijo, y ese número es la forma de pedirlo. Igual que `range()` en el tomo 2, la **cuenta empieza en cero**.

`notas = [4.5, 2.8, 3.9, 5.0, 3.0]`

4.5	2.8	3.9	5.0	3.0
0	1	2	3	4
-5	-4	-3	-2	-1

FIG. 1.1 — LA DOBLE NUMERACIÓN. Arriba, el índice normal: cuenta desde el principio y empieza en 0. Abajo, el índice negativo: cuenta desde el final y empieza en -1. Las dos direcciones apuntan a la misma casilla; usas la que te quede más cerca.

● `indices.py`

```
1 notas = [4.5, 2.8, 3.9, 5.0, 3.0]
2
3 print(notas[0])    # la primera: 4.5
4 print(notas[2])    # la tercera: 3.9  (;cuidado: la 2 es la tercera!)
5 print(notas[-1])   # la última, sin saber cuántas hay: 3.0
6 print(notas[-2])   # la penúltima: 5.0
```

TRAZA PASO A PASO · QUÉ PIDE Y QUÉ RECIBE

ESCRIBES	PYTHON LO LEE COMO	DEVUELVE	POR QUÉ
<code>notas[0]</code>	casilla 0	4.5	La cuenta arranca en cero, no en uno.
<code>notas[4]</code>	casilla 4	3.0	Hay 5 elementos, así que la última casilla es la 4.
<code>notas[5]</code>	casilla 5	IndexError	Esa casilla no existe. Es el error estrella del nivel.
<code>notas[-1]</code>	última casilla	3.0	Los negativos cuentan hacia atrás desde el final.

PELIGRO REAL · EL ERROR DE UNA UNIDAD, OTRA VEZ

Una lista de 5 elementos tiene índices del 0 al 4. Si escribes `notas[5]`, Python detiene el programa en seco:

```
Traceback (most recent call last):
  File "indices.py", line 3, in <module>
    IndexError: list index out of range
```

Traducción: «te pediste una casilla que no existe». Es el mismo *off-by-one* del tomo 2, con otra cara. Regla de supervivencia: **el último índice válido siempre es `len(lista) - 1`**; y si lo que quieres es el último, no calcules nada: escribe `lista[-1]`.

1.4 Medir la lista: `len()`

`len()` devuelve **cuántos elementos hay**, no el último índice. Esa distinción de una unidad es la que separa un programa correcto de uno que se cae en la última vuelta.

```
medir.py

1 materias = ["Matemáticas", "Español", "Física"]
2
3 cuantas = len(materias)
4 print(f"Tienes {cuantas} materias.")
5 print(f"La última es {materias[cuantas - 1]}") # len - 1
6 print(f"La última es {materias[-1]}")         # lo mismo, más limpio
7
8 if len(materias) == 0:                         # ¿la lista está vacía?
9     print("No hay materias registradas.")
```

ERROR CLÁSICO · CONFUNDIR CANTIDAD CON POSICIÓN

`len(materias)` vale 3, pero `materias[3]` **no existe**. Suena obvio leído aquí y se te va a olvidar la primera vez que lo escribas dentro de un ciclo. Cuando veas un `IndexError`, tu primera sospecha debe ser siempre esta: *alguien pidió la casilla número len*.

PREDICE ANTES DE EJECUTAR

Tienes `colores = ["rojo", "verde", "azul", "amarillo"]`. Escribe en papel qué devuelve cada una de estas cuatro líneas *antes* de probarlas: `colores[1]`, `colores[-2]`, `len(colores)`, `colores[len(colores)]`. La cuarta no devuelve nada: explica por qué en una frase.

IGUAL QUE RANGE()

El inicio se incluye, el fin se excluye. Es la misma regla del nivel 9 del tomo 2, y por la misma razón: así

`lista[0:3]` te da exactamente 3 elementos.

1.5 Rebanadas: cortar un trozo de la lista

Un índice te da **un** elemento. Una **rebanada** (en inglés *slice*) te da **una lista nueva** con un trozo de la original. Se escribe con dos puntos: `lista[inicio:fin]`.

```
dias = ["lun", "mar", "mié", "jue", "vie", "sáb", "dom"]
        0       1       2       3       4       5       6

dias[0:5] -> ['lun', 'mar', 'mié', 'jue', 'vie']      entre semana
              |_____|
              incluye el 0                          corta ANTES del 5

dias[5:]   -> ['sáb', 'dom']                        desde el 5 hasta el final
dias[:2]   -> ['lun', 'mar']                        desde el principio hasta antes del 2
dias[::2]  -> ['lun', 'mié', 'vie', 'dom']          de dos en dos
dias[:]    -> copia completa e independiente
```

FIG. 1.2 — LA REBANADA. Los dos puntos significan «desde... hasta antes de...». Si omites un extremo, Python entiende «desde el principio» o «hasta el final». El tercer número, si aparece, es el paso.

● rebanadas.py

```
1 dias = ["lun", "mar", "mié", "jue", "vie", "sáb", "dom"]
2
3 habiles = dias[0:5]      # del 0 al 4
4 finde = dias[-2:]       # los dos últimos
5 alternos = dias[::2]    # saltando de dos en dos
6 al_reves = dias[::-1]   # paso negativo: invierte
7
8 print(habiles)
9 print(finde)
10 print(alternos)
11 print(al_reves)
```

SALIDA EN PANTALLA

```
['lun', 'mar', 'mié', 'jue', 'vie']  
['sáb', 'dom']  
['lun', 'mié', 'vie', 'dom']  
['dom', 'sáb', 'vie', 'jue', 'mié', 'mar', 'lun']
```

TIP · LA REBANADA NUNCA SE CAE

Si pides `dias[3]` y no existe, revienta con `IndexError`. Pero si pides `dias[3:99]`, Python **no protesta**: te devuelve lo que haya y ya. Por eso, cuando solo quieres «los primeros tres, si es que hay tres», `lista[:3]` es más seguro que tres accesos por índice.

1.6 Tres operaciones que ya conoces de los textos

Las listas y las cadenas de texto son primas: ambas son **secuencias**. Por eso comparten operadores, y lo que aprendiste con `str` se aplica igual.

OPERACIÓN	EJEMPLO	QUÉ HACE
Unir <code>+</code>	<code>[1, 2] + [3] → [1, 2, 3]</code>	Crea una lista nueva pegando las dos. No modifica ninguna.
Repetir <code>*</code>	<code>[0] * 5 → [0, 0, 0, 0, 0]</code>	Crea una lista con el mismo valor repetido. Útil para preparar marcadores.
Pertenencia <code>in</code>	<code>"Física" in materias → True</code>	Pregunta si un valor está dentro. Devuelve <code>True</code> o <code>False</code> .
Recorrer <code>for</code>	<code>for nota in notas:</code>	Visita los elementos uno por uno. Es el nivel 3 completo.

EN LA VIDA REAL · LISTAS POR TODAS PARTES

Tu **playlist** es una lista: tiene orden, admite repetidos y puedes saltar a la canción número 7. Cuando arrastras una canción hacia arriba, la app cambia índices.

El **feed de cualquier red social** es una lista de publicaciones que llega ordenada del servidor; lo que ves en pantalla es un ciclo recorriéndola.

El **carrito de compras**, el historial del navegador, la fila de impresión y los mensajes de un chat: todos son listas. Si algo tiene *orden* y puede *crecer*, es una lista.

¿SABÍAS QUÉ?

El ancestro de la lista se llama **arreglo** (*array*) y es tan viejo como los computadores: apareció en Fortran, en 1957. La razón de que los índices empiecen en cero no fue una decisión estética, sino **aritmética de memoria**: la dirección de un elemento se calcula como *inicio + índice × tamaño*. Con el primer elemento en el índice 0, esa multiplicación da cero y el elemento cae justo en el inicio. Empezar en 1 obligaría a restar uno en cada acceso, millones de veces por segundo.

Los arreglos clásicos tenían un tamaño fijo declarado de antemano: si decías 100, eran 100 para siempre. La lista de Python **crece sola**, y esa comodidad —que hoy te parece obvia— fue durante décadas un lujo que había que programar a mano.

■ LABORATORIO DE LA SESIÓN · «TU PRIMERA CAJA»

25 minutos · consola interactiva abierta · escribe cada paso a mano

1. Crea `peliculas` con cinco títulos que hayas visto este año.
2. Imprime la primera y la última **sin contar cuántas hay** (pista: índice negativo).
3. Imprime `len(peliculas)` y luego, a propósito, pide `peliculas[len(peliculas)]`. Lee el mensaje de error completo y escríbelo en tu cuaderno.
4. Sacar una rebanada con las tres del medio.
5. Crea `favoritas = peliculas[:2] + ["Otra más"]` e imprime las dos listas. Comprueba que `peliculas` **no cambió**: unir crea una lista nueva.
6. Responde por escrito: ¿qué imprime `"Otra más" in peliculas` y por qué?

Entrega: un archivo `lab1.py` que se ejecute de arriba abajo sin errores (salvo el del paso 3, que debes dejar comentado con la explicación al lado).

MISIÓN DEL NIVEL 1

«La caja que reemplaza cinco variables»

DIFICULTAD ●●○○○ · 30 MINUTOS · NECESITAS: EDITOR ABIERTO

1. Reescribe `a_la_fuerza.py` con una lista. Calcula el promedio usando `sum(notas) / len(notas)` y comprueba que da lo mismo.
2. Crea una lista con los nombres de tus siete días de la semana e imprime, en tres líneas: el primero, el último y los del fin de semana usando una rebanada.
3. Prepara un marcador de asistencia de 10 días con `[False] * 10` e imprime su longitud.
4. Escribe un programa que pida una palabra con `input()`, la convierta en lista de letras con `list()` y muestre la letra del medio. Pruébalo con una palabra par y una impar: una de las dos te va a incomodar. Explica por qué en dos líneas.
5. **Rómpelo a propósito:** pide un índice al usuario y accede a la lista sin validar. Escribe `99` y lee el error. Después arréglalo con un `if` que compare contra `len()`.

NIVEL JEFE: sin usar `sorted()` ni ningún método nuevo, imprime las tres notas más altas de una lista de cinco. Solo tienes índices, rebanadas y comparaciones. Vas a sufrir un poco — y en el nivel 2 vas a descubrir que existía una línea que lo resuelve. Ese contraste es el objetivo del ejercicio.

Recompensa desbloqueada: *colección*. Tus datos ya no viven sueltos: viven juntos y numerados.

CHECKPOINT · NIVEL 1

1. Una lista tiene 8 elementos. ¿Cuál es el índice del primero, cuál el del último y cuánto vale `len()`?
2. ¿Qué diferencia hay entre `lista[2]` y `lista[2:3]`? (Pista: mira el *tipo* de lo que devuelve cada uno.)
3. Explica con tus palabras por qué `lista[99]` se cae pero `lista[3:99]` no.

ESTE NIVEL EN 3 IDEAS

- Una lista es una sola variable con muchas casillas ordenadas y numeradas desde **cero**.
- El último índice válido es `len - 1`; para el último elemento, usa `[-1]`.
- Una rebanada `[inicio:fin]` devuelve una **lista nueva**: incluye el inicio y excluye el fin.

ROJO

Escribo la lista, pero me pierdo cada vez que tengo que contar posiciones.

ÁMBAR

Indexo bien, aunque todavía dudo con los negativos y las rebanadas.

VERDE

Predigo qué devuelve cualquier índice o rebanada sin ejecutar el código.

Crecer, encoger, buscar y ordenar

Una lista que no cambia es una foto. Aquí aprendes a convertirla en algo que se mueve: la diferencia entre guardar datos y administrarlos.

LO QUE DESBLOQUEARÁS

- 1 Modificar un elemento por su índice y explicar qué significa **mutable**.
- 2 Hacer crecer y encoger una lista con los métodos que se usan de verdad.
- 3 Buscar dentro de una lista sin recorrerla a mano.
- 4 Ordenar de dos formas distintas y saber exactamente cuál usar.

PALABRAS CLAVE

mutable método

.append() .insert()

.remove() .pop() del

.index() .count()

.sort() sorted()

alias .copy()

PALABRA NUEVA: MÉTODO

Un **método** es una acción que pertenece a un objeto y se llama con un punto:

```
lista.append(x) .
```

Ya usaste varios sin saberlo:

```
texto.upper() ,
texto.isdigit() .
```

2.1 Mutable: la palabra que define a las listas

Un texto no se puede cambiar por dentro. Si escribes `nombre[0] = "J"`, Python te detiene. Una lista sí: es **mutable**, es decir, *modificable después de creada*.

mutar.py

```
1 notas = [4.5, 2.8, 3.9]
2
3 notas[1] = 3.5           # el profe subió la nota de la casilla 1
4 print(notas)             # [4.5, 3.5, 3.9] -> la MISMA lista, cambiada
5
6 nombre = "ana"
7 nombre[0] = "A"         # TypeError: los textos son INMUTABLES
```

CONCEPTO CLAVE

Mutable: se puede modificar por dentro sin crear nada nuevo. Las *listas* lo son.

Inmutable: una vez creado, no se toca; para «cambiarlo» hay que fabricar otro. Los *textos*, los *números* y las *tuplas* (nivel 4) lo son.

2.2 Hacer crecer la lista

`.append()` es, con diferencia, el método de lista que más vas a escribir en tu vida. Aprende ese primero y bien.

• `crecer.py`

```
1 compras = ["pan", "leche"]
2
3 compras.append("huevos")           # añade AL FINAL. Uno solo.
4 compras.insert(0, "café")          # añade en la posición 0; el resto se corre
5 compras.extend(["arroz", "sal"])    # añade VARIOS de una lista
6
7 print(compras)
8 print(len(compras))
```

SALIDA EN PANTALLA

```
['café', 'pan', 'leche', 'huevos', 'arroz', 'sal']
6
```

ERROR CLÁSICO • APPEND FRENTE A EXTEND

`compras.append(["arroz", "sal"])` no agrega dos productos: agrega *un* elemento que resulta ser una lista, y te queda `['pan', 'leche', ['arroz', 'sal']]`. Cuando después recorras la lista te encontrarás con una lista dentro de otra sin haberlo pedido. Regla simple: **append** mete una cosa; **extend** vacía otra lista dentro de la tuya.

PELIGRO REAL • EL MÉTODO QUE DEVUELVE NONE

Este es el error más frecuente del nivel entero, y produce un programa que muere sin explicar por qué:

```
1 compras = compras.append("pan")    # ← MAL. compras ahora vale
   None
2 compras.append("pan")
   # ← BIEN. La lista se modifica sola
```

Los métodos que **modifican** la lista (`append`, `insert`, `sort`, `reverse` ...) no devuelven nada: devuelven `None`. Si asignas ese resultado, destruyes tu lista y el error real aparecerá tres líneas más abajo como `AttributeError: 'NoneType' object has no attribute 'append'`. Si el método cambia la lista, no lo pongas a la derecha de un igual.

2.3 Hacer encoger la lista

Hay tres formas de quitar, y se diferencian en *qué sabes* del elemento que quieres fuera:

● encoger.py

```
1 compras = ["café", "pan", "leche", "huevos", "pan"]
2
3 compras.remove("pan")      # por VALOR: quita solo el primero que encuentre
4 ultimo = compras.pop()     # por POSICIÓN: saca el último Y te lo devuelve
5 primero = compras.pop(0)   # pop con índice: saca el de esa casilla
6 del compras[0]             # por POSICIÓN, sin devolver nada
7
8 print(compras, "| saqué:", ultimo, primero)
9 compras.clear()            # vaciar del todo
10 print(compras)
```

SALIDA EN PANTALLA

```
['leche'] | saqué: pan café
[]
```

TIP · POP() ES «QUITAR Y QUEDARME CON ESO»

Cuando programes un juego de cartas, una cola de turnos o un carrito, casi siempre quieres las dos cosas a la vez: sacar el elemento y usarlo. Eso es exactamente `pop()`. Con `del` el elemento desaparece sin que puedas verlo.

ERROR CLÁSICO · QUITAR ALGO QUE NO ESTÁ

`compras.remove("azúcar")` cuando no hay azúcar lanza `ValueError: list.remove(x): x not in list`. La defensa cabe en una línea y es la misma de siempre: **pregunta antes de actuar**.

```
1 if "azúcar" in compras:
2     compras.remove("azúcar")
3 else:
4     print("Ese producto no está en la lista. Escribe uno de:",
        compras)
```

2.4 Buscar dentro de la lista

● buscar.py

```
1 equipo = ["Ana", "Luis", "Sara", "Luis"]
2
3 print("Sara" in equipo)      # True  -> ¿está?
4 print(equipo.index("Sara"))  # 2     -> ¿en qué casilla?
5 print(equipo.count("Luis"))  # 2     -> ¿cuántas veces?
6
7 # .index() se cae si no está: pregunta primero
8 if "Pedro" in equipo:
9     print(f"Pedro está en la posición {equipo.index('Pedro')}")
10 else:
11     print("Pedro no está inscrito en el equipo.")
```

TRES PREGUNTAS DISTINTAS, TRES HERRAMIENTAS

`in` responde ¿sí o no? · `.index()` responde ¿dónde? · `.count()` responde ¿cuántas veces? Elegir mal no da error: da respuestas que no sirven.

LA CONFUSIÓN DEL NIVEL

`.sort()` cambia tu lista
y devuelve `None`.
`sorted()` no la toca y
devuelve una lista nueva. La
diferencia se paga cara.

2.5 Ordenar: dos herramientas parecidas y muy distintas

● ordenar.py

```
1 notas = [3.9, 4.5, 2.8, 5.0]
2
3 copia_ordenada = sorted(notas)      # lista NUEVA; 'notas' intacta
4 print(copia_ordenada, notas)
5
6 notas.sort()                        # ordena la lista ORIGINAL
7 print(notas)
8
9 notas.sort(reverse=True)            # de mayor a menor
10 print(notas)
11
12 notas.reverse()                     # solo da la vuelta; NO ordena
13 print(notas)
```

SALIDA EN PANTALLA

```
[2.8, 3.9, 4.5, 5.0] [3.9, 4.5, 2.8, 5.0]
[2.8, 3.9, 4.5, 5.0]
[5.0, 4.5, 3.9, 2.8]
[2.8, 3.9, 4.5, 5.0]
```

QUIERO...	USO	QUÉ LE PASA A MI LISTA ORIGINAL
Ordenarla de verdad	<code>lista.sort()</code>	Cambia. El orden anterior se pierde para siempre.
Verla ordenada sin tocarla	<code>sorted(lista)</code>	Intacta. Recibes una copia ordenada aparte.
De mayor a menor	<code>sorted(lista, reverse=True)</code>	Intacta. El parámetro también existe en <code>.sort()</code> .
Solo invertir el orden actual	<code>lista.reverse()</code>	Cambia. Ojo: invertir <i>no</i> es ordenar.

PREDICE ANTES DE EJECUTAR

¿Qué imprime `print(sorted(["banano", "Ana", "cereza", "10"]))` ? Antes de ejecutarlo, escribe tu respuesta. Pista: los textos se ordenan por su **código de carácter**, y las mayúsculas van antes que las minúsculas. Cuando lo veas, ya sabrás por qué en las apps reales se ordena con `key=str.lower`.

2.6 El peligro invisible: dos nombres, una sola lista

Este es el concepto más importante del nivel y el que casi nadie explica a tiempo. Cuando escribes `b = a` con listas, **no estás copiando nada**: le estás poniendo un segundo nombre a la misma caja.

```

a = [1, 2, 3]
b = a                b = a.copy()

a ┌───┐
  │   │ ──> [1, 2, 3]
a └───┘
b └───┘

a ──> [1, 2, 3]
b ──> [1, 2, 3] (otra caja)

b.append(4)          b.append(4)
-> a vale [1,2,3,4]  -> a sigue valiendo [1,2,3]
¡sin que lo pidas!

```

FIG. 2.1 — ALIAS CONTRA COPIA. El signo igual copia la *etiqueta*, no el contenido. Para obtener una caja nueva de verdad usa `.copy()` o la rebanada `[:]`.

• alias.py

```
1 originales = [10, 20, 30]
2 alias = originales           # MISMA lista, otro nombre
3 copia = originales.copy()    # lista NUEVA e independiente
4
5 alias.append(40)
6 copia.append(99)
7
8 print(originales)           # [10, 20, 30, 40] ← el alias la modificó
9 print(copia)                # [10, 20, 30, 99] ← la copia va por su cuenta
```

PELIGRO REAL • «MI LISTA CAMBIÓ SOLA»

Cuando pases listas a funciones (tomo 4) o guardes «una copia de seguridad» antes de modificar, este detalle decide si tu respaldo existe o es una ilusión. **Regla:** si vas a modificar y quieres conservar el original, escribe `copia = lista.copy()`. Nunca `copia = lista`.

PELIGRO REAL • BORRAR MIENTRAS RECORRES

Modificar una lista *durante* el ciclo que la recorre produce resultados imposibles de depurar: Python avanza el índice interno mientras los elementos se corren hacia la izquierda, así que **salta elementos en silencio**.

```
1 for n in numeros:           # ← MAL si dentro haces
    numeros.remove(n)
2 for n in numeros.copy():
    # ← BIEN: recorre una foto, modifica el original
```

La otra solución, más elegante y la que usarás en el nivel 3, es no borrar nada: **construir una lista nueva solo con lo que sí quieres conservar**.

2.7 Chuleta de métodos de lista

MÉTODO	EJEMPLO	QUÉ HACE Y QUÉ DEVUELVE
<code>.append(x)</code>	<code>l.append(7)</code>	Añade uno al final. Devuelve <code>None</code> .
<code>.insert(i, x)</code>	<code>l.insert(0, 7)</code>	Inserta en la posición <i>i</i> y corre el resto. Devuelve <code>None</code> .
<code>.extend(otra)</code>	<code>l.extend([8, 9])</code>	Añade todos los elementos de otra colección.
<code>.remove(x)</code>	<code>l.remove("pan")</code>	Quita la primera aparición de ese valor. Se cae si no está.
<code>.pop(i)</code>	<code>turno = l.pop(0)</code>	Quita por posición (por defecto la última) y devuelve el elemento .
<code>.clear()</code>	<code>l.clear()</code>	Vacía la lista y la deja en <code>[]</code> .
<code>.index(x)</code>	<code>l.index("Sara")</code>	Devuelve la posición de la primera aparición. Se cae si no está.
<code>.count(x)</code>	<code>l.count("Luis")</code>	Devuelve cuántas veces aparece. Si no está, devuelve 0.
<code>.sort()</code>	<code>l.sort(reverse=True)</code>	Ordena la propia lista . Devuelve <code>None</code> .
<code>.reverse()</code>	<code>l.reverse()</code>	Invierte el orden actual. No ordena.
<code>.copy()</code>	<code>respaldo = l.copy()</code>	Devuelve una lista nueva con los mismos elementos.

EN LA VIDA REAL • LISTAS QUE SE MUEVEN

La cola de tu app de música: «reproducir a continuación» es `insert(1, cancion)`; «agregar a la cola» es `append`; y cuando termina una canción, la app hace `pop(0)`.

El botón «deshacer» de cualquier editor guarda cada acción en una lista y hace `pop()` para retroceder. A esa estructura se le llama *pila*: lo último que entra es lo primero que sale.

Ordenar por precio, por fecha o por calificación en una tienda es literalmente un `sorted(productos, key=...)` ejecutándose en un servidor a miles de kilómetros.

¿SABÍAS QUÉ?

El `.sort()` de Python usa un algoritmo llamado **Timsort**, inventado en 2002 por Tim Peters —el mismo programador que escribió el *Zen de Python*, ese poema que aparece si escribes `import this`—.

Su idea fue tan buena que hoy es el ordenamiento por defecto de Java y de Android: aprovecha que los datos del mundo real casi nunca están totalmente desordenados (una lista de fechas o de nombres ya suele venir con trozos en orden) y en vez de ignorarlo, **detecta esos trozos ya ordenados y los fusiona**. Cuando escribes una línea de nueve caracteres, estás llamando a veinte años de trabajo de gente muy lista. Esa es la razón real por la que nadie ordena a mano.

■ LABORATORIO DE LA SESIÓN · «LISTA DE COMPRAS QUE FUNCIONA»

30 minutos · un archivo `lab2.py` · ejecuta después de cada paso

1. Empieza con `compras = []` y agrega cuatro productos con `append`.
2. Inserta `"urgente: pilas"` al principio.
3. Pide un producto con `input()` y agrégalo, **pero solo si no está ya** en la lista. Si está, muestra un mensaje con las tres piezas del tomo 2: qué pasó, qué se esperaba, qué hacer.
4. Pide un producto a eliminar y quítalo con `remove`, protegiéndolo con `in`. Prueba a escribir algo que no exista: tu programa **no** debe caerse.
5. Muestra la lista ordenada alfabéticamente **sin modificarla**, y después la lista original, para demostrar en pantalla la diferencia entre `sorted()` y `.sort()`.
6. Haz `respaldo = compras`, vacía `compras` con `clear()` e imprime el respaldo. Explica el desastre en una frase y arréglalo.

Entrega: el programa funcionando + tres líneas escritas explicando el punto 6. Ese punto 6 es el que te vas a encontrar en un trabajo real.

MISIÓN DEL NIVEL 2

«La lista que administra»

DIFICULTAD ●●●○○ · 35 MINUTOS · ENTREGA: 4 PROGRAMAS

1. Gestor de turnos: una lista de nombres; el programa atiende con `pop(0)` e imprime «Atendiendo a X · quedan N en la fila» hasta vaciarla.
2. Top 3: pide cinco números, ordénalos de mayor a menor e imprime solo los tres primeros con una rebanada. Compáralo con lo que sufriste en el nivel jefe del nivel 1.
3. Sin repetidos: partiendo de una lista con duplicados, construye **otra** lista que solo tenga los valores que aparecen una vez. No borres nada: agrega a una lista nueva.
4. Deshacer: guarda cada acción del usuario en una lista `historial` y permite deshacer la última con `pop()`, mostrando qué se deshizo.
5. **Rómpelo a propósito:** escribe `lista = lista.sort()`, ejecútalo e imprime la lista después. Anota el error exacto que aparece en la línea siguiente y explica de dónde salió el `None`.

NIVEL JEFE: escribe un «podio» que reciba una lista de nombres y una lista de puntajes en el mismo orden, y muestre los tres primeros lugares. Vas a necesitar ordenar una lista *en función de la otra*, y con lo que sabes hoy solo se puede a la fuerza bruta. Hazlo así, y anota lo incómodo que fue: en el nivel 6 lo resolverás en tres líneas y entenderás por qué existen las estructuras anidadas.

Recompensa desbloqueada: *administración*. Tus listas ya no solo guardan: crecen, encogen y se ordenan.

CHECKPOINT · NIVEL 2

1. ¿Qué devuelve `lista.append(5)`? ¿Por qué eso hace peligroso escribir `lista = lista.append(5)`?
2. Explica la diferencia entre `.sort()` y `sorted()` como si se lo contaras a alguien que nunca programó.
3. ¿Por qué `b = a` no es una copia? ¿Cómo se hace una copia de verdad?

ESTE NIVEL EN 3 IDEAS

- Las listas son **mutables**: los métodos las cambian por dentro y devuelven `None`.
- Antes de `remove` o `index`, pregunta con `in`: así tu programa no se cae.
- `b = a` crea un **alias**, no una copia. Para copiar: `a.copy()`.

ROJO

Uso `append`, pero mezclo los demás métodos y a veces mi lista queda en `None`.

ÁMBAR

Agrego y quito bien, aunque todavía dudo entre `sort` y `sorted`.

VERDE

Elijo el método correcto a la primera y sé cuándo necesito `.copy()`.

Listas y ciclos: la pareja que resuelve casi todo

El tomo 2 te dio el motor. El nivel 1 te dio el combustible. Aquí los conectas, y por fin los cuatro patrones tienen sobre qué trabajar.

LO QUE DESBLOQUEARÁS

- 1 Recorrer una lista con `for` sin usar un solo número.
- 2 Aplicar los cuatro patrones del tomo 2 a colecciones reales.
- 3 Construir listas nuevas dentro de un bucle: filtrar y transformar.
- 4 Leer y escribir una **comprensión de lista** sencilla.

PALABRAS CLAVE

`for ... in` `sum()` `max()`
`min()` `enumerate()`
`range(len())` `filtrar`
`transformar` `comprensión`
`lista acumuladora`

DEL TOMO 2

Ya recorriste un texto letra por letra con `for letra in palabra`. Una lista se recorre *exactamente igual*. No hay sintaxis nueva: hay una idea nueva.

3.1 Recorrer: el `for` que no cuenta

Un `for` sobre una lista te entrega los **elementos**, no sus posiciones. No hace falta `range`, ni saber cuántos hay, ni tocar índices.

● recorrer.py

```
1 materias = ["Matemáticas", "Español", "Física"]
2
3 for materia in materias:           # 'materia' vale un elemento distinto por vuelta
4     print(f"Cursando: {materia}")
5
6 print("Fin del recorrido.")        # fuera del bucle
```

SALIDA EN PANTALLA

```
Cursando: Matemáticas
Cursando: Español
Cursando: Física
Fin del recorrido.
```

TIP · NOMBRA LA VARIABLE EN SINGULAR

`for nota in notas , for alumno in alumnos , for producto in productos .`
Esa pareja plural/singular hace que la línea se lea como una frase en español y elimina el 90 % de la confusión sobre «qué contiene qué».

ERROR CLÁSICO · CONFUNDIR EL ELEMENTO CON LA LISTA

Dentro del bucle, `materia` es un texto; `materias` sigue siendo la lista completa. Escribir `print(materias)` dentro del ciclo imprime la lista entera tres veces. Si tu salida repite todo en cada vuelta, mira si te sobró una s.

REGLA PROFESIONAL

Si Python ya trae la herramienta, úsala. Escribir a mano lo que existe hecho no te hace mejor programador: te da más superficie para equivocarte.

3.2 Lo que ya no tienes que programar

Antes de escribir un bucle, mira si la respuesta cabe en una llamada. Estas cuatro funciones integradas trabajan sobre cualquier lista de números:

● `integradas.py`

```
1 notas = [4.5, 2.8, 3.9, 5.0, 3.0]
2
3 print(len(notas))           # 5      cuántas hay
4 print(sum(notas))           # 19.2    la suma total
5 print(max(notas))           # 5.0     la mayor
6 print(min(notas))           # 2.8     la menor
7 print(sum(notas) / len(notas)) # 3.84  el promedio
```

Compáralo con lo que costaba en el tomo 2: cinco `input()`, un acumulador, un contador y un campeón. **Y aun así necesitas saber hacerlo a mano**, por dos razones: porque en cuanto la condición se complica («solo las aprobadas») las funciones integradas ya no sirven solas, y porque entender el patrón es lo que te permitirá inventar el que no existe.

PELIGRO REAL · LA LISTA VACÍA

`sum([])` vale 0 sin quejarse, pero `max([])` lanza `ValueError: max() arg is an empty sequence` y `sum(notas) / len(notas)` revienta con `ZeroDivisionError`. Es el mismo caso borde del tomo 2: «ninguno» también es un resultado posible. Protege siempre con `if len(notas) > 0:` antes de calcular un promedio.

3.3 Los cuatro patrones, ahora sobre listas

Son los mismos cuatro moldes del nivel 10 del tomo 2, con una diferencia enorme: ya no hay que pedirle los datos al usuario uno por uno. Los datos **ya están**, y el bucle solo los visita.

1 • ANTES DEL BUCLE

preparas la variable

2 • DENTRO DEL BUCLE

la actualizas en cada vuelta

3 • DESPUÉS DEL BUCLE

la usas o la muestras

● cuatro_patrones.py

```

1 notas = [4.5, 2.8, 3.9, 5.0, 3.0]
2 MINIMA = 3.0
3
4 suma = 0          # acumulador
5 aprobadas = 0     # contador
6 mejor = notas[0]  # campeón: el PRIMER dato real, no un 0 inventado
7 hubo perfecta = False # bandera
8
9 for nota in notas:
10     suma += nota
11     if nota >= MINIMA:
12         aprobadas += 1
13     if nota > mejor:
14         mejor = nota
15     if nota == 5.0:
16         hubo perfecta = True
17
18 print(f"Promedio: {suma / len(notas):.2f}")
19 print(f"Aprobadas: {aprobadas} de {len(notas)}")
20 print(f"Mejor nota: {mejor}")
21 print(f"¿Alguna 5.0? {'Sí' if hubo perfecta else 'No'}")

```

TRAZA PASO A PASO • CUATRO VARIABLES A LA VEZ

VUELTA	NOTA	SUMA	APROBADAS	MEJOR	HUBO_PERFECTA
inicio	—	0	0	4.5	False
1. ^a	4.5	4.5	1	4.5	False
2. ^a	2.8	7.3	1	4.5	False
3. ^a	3.9	11.2	2	4.5	False
4. ^a	5.0	16.2	3	5.0	True
5. ^a	3.0	19.2	4	5.0	True

Míralo **columna por columna**, no fila por fila: cada columna es un patrón viviendo su propia vida. Fíjate en la fila «inicio»: el campeón arranca en `notas[0]`, no en cero. Ese detalle es lo que evita que un programa de temperaturas bajo cero conteste «máxima: 0».

PREDICE ANTES DE EJECUTAR

Cambia la lista por `[-4, -9, -2]` y pon `mejor = 0` en lugar de `mejor = notas[0]`. ¿Qué imprimirá como mejor valor? Escribe tu respuesta, ejecútalo y explica en una frase por qué ese resultado es *peor* que un error: el programa no se cae, simplemente miente.

CUÁNDO SÍ NECESITAS EL ÍNDICE

Para *mostrar* la posición al usuario, para *modificar* la lista en el sitio o para *comparar* un elemento con el siguiente. Fuera de esos casos, no lo uses.

3.4 Cuando además necesitas la posición: `enumerate()`

A veces no basta con el valor: quieres decir «la nota número 3». La solución de principiante es `range(len(lista))`; la solución que usa todo el mundo es `enumerate()`, que te entrega la **posición y el valor a la vez**.

• posiciones.py

```
1 materias = ["Matemáticas", "Español", "Física"]
2
3 # Forma incómoda: solo tienes el índice y hay que ir a buscar el valor
4 for i in range(len(materias)):
5     print(i, materias[i])
6
7 # Forma buena: los dos a la vez, y empezando a contar en 1 para el humano
8 for numero, materia in enumerate(materias, start=1):
9     print(f"{numero}. {materia}")
```

SALIDA EN PANTALLA

```
0 Matemáticas
1 Español
2 Física
1. Matemáticas
2. Español
3. Física
```

CONCEPTO CLAVE

`enumerate(lista)` entrega en cada vuelta **una pareja**: posición y elemento. Por eso el `for` lleva dos nombres separados por coma. El parámetro `start=1` solo cambia por dónde empieza a contar el número que *muestras*: el índice real de la lista sigue empezando en 0.

3.5 El patrón estrella: construir una lista nueva

Este es el patrón que más vas a usar el resto de tu vida como programador, y es un acumulador con otra ropa: en vez de sumar números, **acumula elementos**. Sirve para las dos operaciones fundamentales de los datos: **filtrar** (quedarme con algunos) y **transformar** (cambiarlos todos).

● filtrar_y_transformar.py

```
1 notas = [4.5, 2.8, 3.9, 5.0, 1.5]
2
3 # FILTRAR: me quedo solo con las que cumplen
4 aprobadas = [] # lista vacía ANTES del bucle
5 for nota in notas:
6     if nota >= 3.0:
7         aprobadas.append(nota) # acumulo el elemento
8
9 # TRANSFORMAR: cambio todos por su versión nueva
10 sobre_100 = []
11 for nota in notas:
12     sobre_100.append(nota * 20)
13
14 print(aprobadas)
15 print(sobre_100)
```

SALIDA EN PANTALLA

```
[4.5, 3.9, 5.0]
[90.0, 56.0, 78.0, 100.0, 30.0]
```

BUENA PRÁCTICA · CONSTRUIR EN VEZ DE BORRAR

¿Recuerdas el peligro del nivel 2 (borrar mientras recorres)? Este patrón lo elimina de raíz: **no toques la lista original, fabrica otra**. Es más fácil de leer, imposible de romper, y además conservas los datos originales por si los necesitas después.

SIN MIEDO

Una comprensión no es magia ni un tema avanzado: es *exactamente* el bucle de arriba, escrito de derecha a izquierda en una línea.

3.6 Comprensión de listas: el mismo bucle en una línea

Como filtrar y transformar es tan común, Python trae una forma corta de escribirlo. Se llama **comprensión de lista** y verás una en cualquier código profesional que abras.

```
sobre_100 = [ nota * 20 for nota in notas if nota >= 3.0 ]
```

qué guardo
en cada
vuelta

de dónde saco
los valores

(opcional)
me quedo solo
con los que cumplen

Se lee al revés: primero el for, luego el if, y al final lo que se guarda.

FIG. 3.1 — ANATOMÍA DE UNA COMPRENSIÓN. Las tres piezas son las mismas del bucle largo, reordenadas. La lista vacía y el `append` desaparecen porque Python los pone por ti.

● comprehension.py

```
1 notas = [4.5, 2.8, 3.9, 5.0, 1.5]
2
3 aprobadas = [n for n in notas if n >= 3.0]      # filtrar
4 sobre_100 = [n * 20 for n in notas]            # transformar
5 cuadrados = [i ** 2 for i in range(1, 6)]       # también con range
6 mayusculas = [m.upper() for m in ["ana", "luis"]] # también con textos
7
8 print(aprobadas, sobre_100, cuadrados, mayusculas, sep="\n")
```

BUCLE LARGO	COMPRESIÓN	CUÁL CONVIENE
Tres o cuatro líneas: lista vacía, <code>for</code> , <code>if</code> , <code>append</code>	Una línea	La comprensión, si cabe cómoda en una línea.
Admite varias instrucciones por vuelta	Solo una expresión	El bucle largo, cuando dentro pasan varias cosas.
Fácil de depurar con un <code>print</code> espía	Difícil de espiar por dentro	El bucle largo mientras estás depurando.

ERROR CLÁSICO · LA COMPRESIÓN ILEGIBLE

Escribir `[x*2 for l in datos for x in l if x>0 and x%3==0]` no te hace avanzado: te hace lento la próxima semana, cuando ni tú entiendas qué hace. **Regla de la casa: si una comprensión no cabe holgada en una línea o lleva más de un `if`, escríbela como bucle normal.** El código se escribe una vez y se lee cincuenta.

EN LA VIDA REAL · FILTRAR Y TRANSFORMAR ES TODO

Un **buscador** filtra millones de registros con la misma idea que tu `if nota >= 3.0`, solo que repartida en cientos de máquinas.

Un **filtro de fotos** es una transformación: recorre los píxeles y devuelve una colección nueva. Nada se «pinta encima»: se construye otra imagen.

El «**solo disponibles**» de una tienda, el «no leídos» de tu correo y el «amigos en común» de una red social son la misma línea de código con otros nombres. Cuando domines este patrón, habrás entendido cómo funciona la mitad del software que usas.

¿SABÍAS QUÉ?

La notación `[x for x in datos if ...]` no la inventó Python: viene de las **matemáticas**. En teoría de conjuntos, «el conjunto de los x tales que x pertenece a D y x cumple P» se escribe casi igual, y esa forma de definir conjuntos se llama *comprensión* (frente a definirlos por *extensión*, enumerando uno por uno).

Python la tomó prestada de un lenguaje llamado **Haskell** y la incorporó en la versión 2.0, en el año 2000. Es uno de los pocos casos en que una notación matemática de hace más de un siglo entró intacta en un lenguaje de programación... y hoy la escriben a diario millones de personas que nunca supieron de dónde salió.

■ LABORATORIO DE LA SESIÓN • «INFORME DE NOTAS 2.0»

35 minutos • un archivo `lab3.py` • ejecuta después de cada paso

1. Pide al usuario cuántas notas va a registrar (válidalo con `.isdigit()`) y llénalas con un `for` y `append` dentro de una lista.
2. Muestra el listado numerado con `enumerate(notas, start=1)`.
3. Calcula promedio, mejor y peor con las funciones integradas. Protege el caso de **cero notas**.
4. Construye dos listas nuevas: `aprobadas` y `perdidas`. Muestra cuántas hay en cada una **sin usar contadores** (pista: `len()`).
5. Reescribe el paso 4 con dos comprensiones de lista y comprueba que el resultado es idéntico.
6. Añade un ranking: muestra las notas ordenadas de mayor a menor **sin destruir** el orden en que fueron ingresadas.

Entrega: el programa + una frase escrita respondiendo: ¿en qué paso te resultó más clara la comprensión y en cuál preferiste el bucle largo?

MISIÓN DEL NIVEL 3

«Los cuatro patrones sueltos»

DIFICULTAD ●●●○○ · 40 MINUTOS · ENTREGA: 5 PROGRAMAS CORTOS

1. Dada una lista de temperaturas (con negativos incluidos), imprime la máxima, la mínima y el promedio. Hazlo **dos veces**: a mano con los patrones y con las funciones integradas. Deben coincidir.
2. Cuenta cuántas palabras de una lista tienen más de 5 letras y muéstralas numeradas.
3. De una lista de precios, construye otra con el precio con IVA (19 %) redondeado a dos decimales.
4. Recorre una lista de nombres y construye una lista con las **iniciales** (pista: `nombre[0]`).
5. **Rómpelo a propósito**: pon `aprobadas = []` *dentro* del bucle. Ejecuta, observa el resultado y explica por qué es exactamente el mismo bug del acumulador reiniciado del tomo 2.

NIVEL JEFE: escribe un detector de duplicados que reciba una lista y devuelva otra con los valores que aparecen más de una vez, **sin repetir el aviso**. Vas a necesitar dos listas: la de resultados y una de «ya avisados». Cuando llegues al nivel 5 descubrirás que un diccionario resuelve esto de una forma mucho más elegante — y ahí entenderás para qué existen.

Recompensa desbloqueada: *procesamiento*. Ya puedes tomar una colección entera y sacarle respuestas.

CHECKPOINT · NIVEL 3

1. ¿Qué contiene la variable del `for` cuando recorres una lista: la posición o el valor?
2. ¿Por qué el campeón debe inicializarse con `lista[0]` y no con `0`?
3. Escribe, con una comprensión, la lista de los números pares del 1 al 20.

ESTE NIVEL EN 3 IDEAS

- `for elemento in lista` te da los **valores**; `enumerate()` te da la posición y el valor.
- Los cuatro patrones del tomo 2 se aplican igual: la variable nace **antes** del bucle.
- Filtrar y transformar = lista vacía + `append`, o una **comprensión** si cabe en una línea legible.

ROJO

Recorro la lista, pero me pierdo cuando hay que acumular en otra.

ÁMBAR

Uso los patrones; las comprensiones todavía las leo con esfuerzo.

VERDE

Convierto cualquier bucle de filtrado en una comprensión y al revés.

Tuplas: lo que no se debe tocar

Una lista a la que le quitaron el botón de editar. Suen a limitación y es justo lo contrario: es una promesa de que ese dato no va a cambiar por accidente.

LO QUE DESBLOQUEARÁS

- 1 Crear tuplas y explicar qué significa **immutable** con un ejemplo propio.
- 2 Decidir sin dudar entre lista y tupla ante un dato nuevo.
- 3 Empaquetar y **desempaquetar** varios valores en una línea.
- 4 Intercambiar dos variables sin variable auxiliar.

PALABRAS CLAVE

tupla () immutable
empaquetar
desempaquetar TypeError
constante registro
intercambio _

LA PREGUNTA QUE DECIDE

«¿Este conjunto de datos va a cambiar durante el programa?» Si la respuesta es no, tienes una tupla. Si es sí, tienes una lista.

4.1 El problema: datos que nadie debería poder cambiar

Piensa en los días de la semana, en los colores de un semáforo, en las coordenadas de un punto o en la fecha de nacimiento de alguien. Son colecciones, sí, pero **tienen algo distinto**: no tiene ningún sentido agregarles un elemento ni cambiarles uno. Si tu programa *puede* hacerlo, algún día lo hará por error.

tuplas.py

```
1 dias = ("lun", "mar", "mié", "jue", "vie", "sáb", "dom")
2 punto = (12, 45)
3 rojo_puro = (255, 0, 0)           # un color en formato RGB
4
5 print(dias[0])                    # se indexa igual que una lista: 'lun'
6 print(dias[-2:])                  # se rebana igual: ('sáb', 'dom')
7 print(len(dias))                  # se mide igual: 7
8 print("mié" in dias)              # se pregunta igual: True
9
10 dias[0] = "LUN"                  # ← aquí Python dice que NO
```

SALIDA EN PANTALLA

```
lun
('sáb', 'dom')
7
True
Traceback (most recent call last):
TypeError: 'tuple' object does not support item assignment
```

CONCEPTO CLAVE

Tupla: una colección *ordenada* e **inmutable**. Se escribe entre paréntesis `( )`. Puedes leerla, recorrerla, indexarla y rebanarla exactamente como una lista; lo único que no puedes es *modificarla*: ni cambiar, ni agregar, ni quitar.

Dicho corto: **una tupla es una lista con la garantía de que nadie la va a tocar.**

ERROR CLÁSICO · LA COMA ES LA QUE MANDA, NO EL PARÉNTESIS

`(5)` **no** es una tupla: es el número 5 entre paréntesis. Una tupla de un solo elemento se escribe `(5,)`, con coma final. Y al revés: `punto = 12, 45` **sí** es una tupla, aunque no lleve paréntesis. La regla real de Python es esa: *lo que crea la tupla es la coma*. Los paréntesis solo sirven para que se lea mejor.

TIP DE LECTURA

Cuando abras código ajeno y veas paréntesis en vez de corchetes, ya sabes el mensaje que te dejó quien lo escribió: «*esto es fijo, no lo toques*».

4.2 Lista o tupla: la tabla que decide

CRITERIO	LISTA <code>[]</code>	TUPLA <code>()</code>
¿Se puede modificar?	Sí (mutable)	No (inmutable)
¿Tiene orden e índices?	Sí	Sí, idénticos
Métodos disponibles	Once o más: <code>append</code> , <code>sort</code> ...	Solo dos: <code>.count()</code> y <code>.index()</code>
Velocidad y memoria	Algo más pesada	Más ligera y algo más rápida
¿Sirve como clave de diccionario?	No	Sí (nivel 5)
Úsala para...	Colecciones que crecen: carrito, notas, turnos, historial	Datos fijos: coordenadas, colores, configuración, fechas, registros

REGLA PRÁCTICA DE BOLSILLO

Lista = «**muchas cosas del mismo tipo**». Diez notas, cien productos, mil usuarios.

Tupla = «**varias partes de una misma cosa**». El `x` y el `y` de un punto; el nombre, la edad y la nota de un estudiante. Si al mirar la colección piensas «esto es *un* dato con varias piezas», es tupla.

4.3 Empaquetar y desempaquetar

Aquí está la razón por la que las tuplas aparecen todo el tiempo en Python, incluso cuando nadie las escribe a propósito: sirven para **mover varios valores como si fueran uno** y luego repartirlos en variables con nombre.

EMPAQUETAR

punto = (12, 45)
└─┬─┘
una sola cosa

DESEMPAQUETAR

x, y = punto
└─┬─┘
dos nombres reciben
cada pieza, en orden

Condición: tantos nombres a la izquierda como elementos haya.
Si no coinciden -> ValueError: not enough values to unpack

FIG. 4.1 — EL PAQUETE Y SUS PIEZAS. Empaquetar junta valores; desempaquetar los reparte. Es la operación que hace que `enumerate()` del nivel anterior funcione.

● desempaquetar.py

```
1 estudiante = ("Ana", 16, 4.5)           # empaquetado
2 nombre, edad, promedio = estudiante     # desempaquetado
3 print(f"{nombre} tiene {edad} años y va en {promedio}")
4
5 # El intercambio más elegante de Python
6 a, b = 10, 20
7 a, b = b, a                             # sin variable auxiliar
8 print(a, b)                             # 20 10
9
10 # El guion bajo: "este valor no me interesa"
11 nombre, _, promedio = estudiante
12
13 # Y esto ya lo usabas sin saberlo: enumerate entrega tuplas
14 for posicion, valor in enumerate(["a", "b"]):
15     print(posicion, valor)
```

EN LA VIDA REAL · EL INTERCAMBIO EN UNA LÍNEA

En casi todos los lenguajes, cambiar dos valores de sitio exige una tercera variable (`temp = a; a = b; b = temp`). En Python es `a, b = b, a` , y no es un truco: primero se construye la tupla `(b, a)` con los valores viejos y después se desempaqueta. Esa línea aparece en el corazón de casi todos los algoritmos de ordenamiento del mundo.

ADELANTO DEL TOMO 4

Cuando aprendas funciones verás `return maximo, minimo` . Eso devuelve una **tupla**, y quien la recibe la desempaqueta. Es la forma estándar de devolver varios resultados.

4.4 Tuplas como registros: la ficha de una cosa

Una lista de tuplas es la forma más simple de guardar una **tabla**: cada tupla es una fila, y cada posición dentro de la tupla es una columna. Es el paso previo a lo que verás en el nivel 6.

● registros.py

```
1 notas = [  
2     ("Matemáticas", 4.5),  
3     ("Español", 2.8),  
4     ("Física", 3.9),  
5 ]  
6  
7 for materia, nota in notas:      # desempaqueta cada fila  
8     estado = "aprobada" if nota >= 3.0 else "perdida"  
9     print(f"{materia:<14} {nota:.1f} {estado}")
```

SALIDA EN PANTALLA

Matemáticas	4.5	aprobada
Español	2.8	perdida
Física	3.9	aprobada

PREDICE ANTES DE EJECUTAR

La lista `notas` es mutable, pero cada tupla de adentro no lo es. ¿Qué crees que pasa con cada una de estas dos líneas?

```
notas.append(("Química", 4.0)) · notas[0][1] = 5.0
```

Escribe tu predicción, ejecútalas y explica la diferencia con las palabras *mutable* e *immutable*.

¿SABÍAS QUÉ?

La palabra *tupla* viene de la terminación de *quintupla*, *séxtupla*, *séptupla*... En matemáticas, una *n*-tupla es una secuencia ordenada de *n* elementos, y esa es exactamente la idea que Python tomó prestada.

La inmutabilidad no es una restricción caprichosa: es lo que permite que una tupla se pueda usar como **clave de un diccionario** (nivel 5) y lo que hace que sea segura de compartir entre distintas partes de un programa. Si un dato no puede cambiar, nadie puede cambiártelo a tus espaldas — y en programas grandes, con muchas piezas tocando los mismos datos, esa garantía vale oro. Los lenguajes modernos van justamente en esa dirección: **hacer immutable todo lo que no necesite cambiar**.

■ LABORATORIO DE LA SESIÓN · «DATOS QUE NO SE TOCAN»

20 minutos · consola interactiva

1. Crea `MESES` como tupla con los doce meses. Imprime el mes número 7 recordando que el índice empieza en cero.
2. Intenta `MESES[0] = "Enero!"` y copia el error exacto en tu cuaderno.
3. Crea `colores = [("rojo", (255,0,0)), ("verde", (0,128,0))]` y recórrela desempaquetando nombre y componentes. Imprime cada color con formato `rojo → R: 255 G:0 B:0`.
4. Escribe `t = (5)` y `t2 = (5,)`. Imprime `type(t)` y `type(t2)` y explica la diferencia en una línea.
5. Guarda tres datos tuyos en una tupla, desempaquétalos en tres variables e imprímelos en una f-string.
6. Convierte una tupla en lista con `list(mi_tupla)`, modifícala y vuelve a convertirla con `tuple(...)`. Acabas de descubrir cómo se «modifica» algo inmutable de verdad: **fabricando uno nuevo**.

Entrega: `lab4.py` y las respuestas escritas de los puntos 2, 4 y 6.

MISIÓN DEL NIVEL 4

«Lo fijo y lo que se mueve»

DIFICULTAD ●●○○○ · 25 MINUTOS · ENTREGA: 4 PROGRAMAS CORTOS

1. Define una tupla `DIAS` y pide al usuario un número del 1 al 7; muestra el día correspondiente y valida el rango con lo aprendido en el tomo 2.
2. Guarda cinco ciudades con sus coordenadas como lista de tuplas y muéstralas alineadas en columnas.
3. Escribe un programa que reciba dos variables y las intercambie con la línea de una sola instrucción; imprime antes y después.
4. Toma la lista de tuplas del punto 2 y ordénala alfabéticamente con `sorted()`. Observa que ordena por el primer elemento de cada tupla y explica por qué eso tiene sentido.
5. **Rómpelo a propósito:** intenta `DIAS.append("otro")` y anota el mensaje. ¿Por qué el error habla de *atributo* y no de *índice*?

NIVEL JEFE: escribe un programa que reciba una lista de tuplas `(nombre, puntaje)` y muestre el podio ordenado **por puntaje de mayor a menor**. Pista: `sorted()` acepta un parámetro `key`, y necesitarás decirle que mire la *segunda* posición de cada tupla. Investiga qué es una función `lambda`, úsala sin entenderla del todo y anota tu duda: la resolverás en el tomo 4.

Recompensa desbloqueada: *integridad*. Ya puedes proteger los datos que no deben cambiar.

CHECKPOINT · NIVEL 4

1. Nombra tres cosas que una tupla puede hacer igual que una lista y dos que no puede hacer.
2. ¿Qué diferencia hay entre `(5)` y `(5,)`?
3. Explica qué ocurre exactamente en la línea `a, b = b, a`.

ESTE NIVEL EN 3 IDEAS

- Una tupla es una colección ordenada e **inmutable**: se lee igual que una lista, pero no se modifica.
- Lo que crea la tupla es **la coma**, no los paréntesis.
- **Desempaquetar** reparte los elementos en variables con nombre; es la base de `enumerate()` y del retorno múltiple.

ROJO

Veo la tupla como «una lista rara» y no sé cuándo usarla.

ÁMBAR

Las creo y las leo, pero el desempaquetado todavía me confunde.

VERDE

Elijo tupla o lista con un argumento claro y desempaqueto sin pensarlo.

Diccionarios: buscar por nombre, no por número

Hasta hoy, para encontrar un dato tenías que saber en qué casilla vivía. Aquí dejas de contar posiciones y empiezas a preguntar por su nombre.

LO QUE DESBLOQUEARÁS

- 1 Crear un diccionario y explicar la pareja **clave** → **valor**.
- 2 Leer datos sin que el programa se caiga, usando `.get()`.
- 3 Agregar, actualizar y borrar entradas con seguridad.
- 4 Recorrer un diccionario con `.items()` y contar cualquier cosa.

PALABRAS CLAVE

diccionario { }

clave valor

par clave-valor

KeyError .get()

.keys() .values()

.items() frecuencia

tabla hash

LA SEÑAL DE ALARMA

Si te encuentras manteniendo **dos listas del mismo largo** donde la posición 3 de una corresponde a la posición 3 de la otra, estás pidiendo a gritos un diccionario.

5.1 El problema: las listas paralelas

Quieres guardar el precio de cada producto. Con listas, la única salida es esta:

● listas_paralelas.py

```
1 productos = ["pan", "leche", "huevos"]
2 precios = [3500, 4200, 9800]
3
4 # ¿Cuánto vale la leche? Hay que buscar su posición... y rezar
5 i = productos.index("leche")
6 print(precios[i])
```

Funciona, y es frágil por tres razones que se pagan caras: si borras un producto y olvidas borrar su precio, **todo se desalinea en silencio**; buscar exige recorrer la lista entera; y el código no dice en ninguna parte que esas dos listas están relacionadas. Lo sabes tú, hoy. En dos semanas, no.

Un **diccionario** guarda la relación explícitamente: cada dato viene *pegado* a su nombre.

● con_diccionario.py

```
1 precios = {
2     "pan": 3500,
3     "leche": 4200,
4     "huevos": 9800,
5 }
6
7 print(precios["leche"]) # 4200 – directo, sin buscar
8 print(len(precios))    # 3 – cuántas parejas hay
```

LISTA

índice	valor
0 ->	"pan"
1 ->	"leche"
2 ->	"huevos"

Tú preguntas: dame el 1
(hay que saber contar)

DICCIONARIO

clave	valor
"pan" ->	3500
"leche" ->	4200
"huevos" ->	9800

Tú preguntas: dame "leche"
(hay que saber el nombre)

FIG. 5.1 — DOS FORMAS DE GUARDAR. La lista te obliga a recordar posiciones; el diccionario te deja preguntar por el nombre. Los dos guardan lo mismo: cambia por dónde entras a buscarlo.

CONCEPTO CLAVE

Diccionario: una colección de parejas **clave** → **valor**. Se escribe entre llaves `{ }`, cada pareja se separa con coma y la clave del valor con dos puntos.

Clave: el nombre por el que buscas. Debe ser *única* e *inmutable* (texto, número o tupla).

Valor: el dato guardado. Puede ser *cualquier cosa*: un número, un texto, una lista, otro diccionario. Y sí puede repetirse.

LA METÁFORA QUE FUNCIONA

Es un diccionario de verdad, el de papel: buscas la **palabra** (la clave) y encuentras la **definición** (el valor). No lo abres por la página 348 esperando encontrar «murciélago»: lo buscas por su nombre. Y ninguna palabra aparece dos veces, aunque muchas compartan definición.

5.2 Leer sin caerse: `[]` frente a `.get()`

Hay dos maneras de pedir un valor, y elegir mal es el error número uno de este nivel.

● leer.py

```
1 alumno = {"nombre": "Ana", "edad": 16, "promedio": 4.2}
2
3 print(alumno["nombre"])           # Ana
4 print(alumno.get("nombre"))       # Ana – lo mismo
5
6 print(alumno.get("correo"))       # None – no existe, pero NO se cae
7 print(alumno.get("correo", "sin registrar")) # valor por defecto
8
9 print(alumno["correo"])           # ← KeyError: el programa muere aquí
```

PELIGRO REAL · KEYERROR

Pedir una clave que no existe con corchetes detiene el programa igual que un `IndexError` :

```
Traceback (most recent call last):
KeyError: 'correo'
```

Y ocurre por cosas tan tontas como una mayúscula: `"Nombre"` y `"nombre"` son **claves distintas**. Las tres defensas, en orden de preferencia: `.get()` con valor por defecto; un `if "clave" in dic:` antes de leer; o normalizar siempre las claves a minúsculas al guardarlas.

SITUACIÓN	USA	POR QUÉ
La clave tiene que existir y si no es un fallo grave	<code>dic["clave"]</code>	Que se caiga ruidosamente es mejor que seguir con datos falsos.
La clave podría no estar (dato opcional)	<code>dic.get("clave", defecto)</code>	Devuelve algo razonable y el programa continúa.
Necesitas decidir según si está o no	<code>if "clave" in dic:</code>	<code>in</code> pregunta por las claves , no por los valores.

TIP CLAVE

Asignar a una clave que no existe la **crea**. Asignar a una que *sí* existe la **reemplaza**. Es la misma línea: por eso hay que tener cuidado con las mayúsculas y los espacios.

5.3 Agregar, actualizar y borrar

● modificar.py

```
1 inventario = {"pan": 10, "leche": 5}
2
3 inventario["huevos"] = 30      # la clave no existía -> se CREA
4 inventario["pan"] = 12        # la clave existía -> se REEMPLAZA
5 inventario["leche"] += 3      # leer, sumar y guardar de un golpe
6
7 inventario.update({"sal": 2, "pan": 15}) # varias a la vez
8
9 retirado = inventario.pop("sal") # borra y DEVUELVE el valor
10 del inventario["huevos"]        # borra sin devolver nada
11
12 print(inventario, "| retiré:", retirado)
13 print("pan" in inventario)      # True - pregunta por la CLAVE
```

SALIDA EN PANTALLA

```
{'pan': 15, 'leche': 8} | retiré: 2
True
```

ERROR CLÁSICO • CLAVES QUE NO SON LO QUE PARECEN

Estas cuatro claves son **cuatro claves distintas**: "pan", "Pan", "pan " (con espacio) y "PAN". Cuando las claves vienen de un `input()`, limpia siempre antes de guardar o consultar: `clave = input("Producto: ").strip().lower()`. Dos métodos, cero bugs.

5.4 Recorrer un diccionario

Un diccionario tiene *dos* cosas por entrada, así que hay tres formas de recorrerlo. La tercera es la que usarás el 90 % de las veces.

● recorrer_dic.py

```
1 precios = {"pan": 3500, "leche": 4200, "huevos": 9800}
2
3 for producto in precios:           # 1. solo CLAVES (lo mismo que .keys())
4     print(producto)
5
6 for valor in precios.values():      # 2. solo VALORES
7     print(valor)
8
9 for producto, valor in precios.items(): # 3. LOS DOS (el que usarás)
10    print(f"{producto:<10} ${valor:,}")
11
12 print(f"Total del carrito: ${sum(precios.values()):,}")
```

SALIDA EN PANTALLA · (SOLO EL TERCER BUCLE Y EL TOTAL)

```
pan          $3,500
leche        $4,200
huevos       $9,800
Total del carrito: $17,500
```

POR QUÉ .ITEMS() LLEVA DOS NOMBRES

Porque en cada vuelta entrega una **tupla** (clave, valor) y tú la **desempaquetas** — exactamente lo que aprendiste en el nivel 4. Todo se conecta: el nivel 4 no era un desvío, era la preparación para esta línea.

PREDICE ANTES DE EJECUTAR

¿Qué imprime `print("leche" in precios)` ? ¿Y `print(4200 in precios)` ? Solo una de las dos da `True` . Escribe cuál y por qué antes de probarlo; si aciertas, entendiste que `in` mira las claves. Para buscar por valor necesitas `in precios.values()` .

EL PATRÓN QUE LO JUSTIFICA TODO

Si alguien te pregunta «¿para qué sirve un diccionario?», este ejemplo es la respuesta más corta y más convincente: contar cosas.

5.5 El patrón estrella: contar cualquier cosa

En el tomo 2 contabas con una variable por cada cosa que querías contar. Eso funciona con dos o tres categorías; con veintiocho letras del alfabeto, es imposible. El diccionario resuelve el problema entero **creando los contadores sobre la marcha**.

• contador_letras.py

```
1 frase = input("Escribe una frase: ").lower()
2 conteo = {} # diccionario vacío ANTES del bucle
3
4 for letra in frase:
5     if letra == " ":
6         continue # los espacios no cuentan
7     conteo[letra] = conteo.get(letra, 0) + 1 # ← la línea clave
8
9 for letra, veces in conteo.items():
10    print(f"{letra}: {'#' * veces} ({veces})")
```

SALIDA EN PANTALLA

Escribe una frase: banana
b: # (1)
a: ### (3)
n: ## (2)

TRAZA PASO A PASO • CÓMO SE VA LLENANDO EL DICCIONARIO

VUELTA	LETRA	CONTEO.GET(LETRA, 0)	CONTEO DESPUÉS
inicio	—	—	{}
1. ^a	b	0 (no existe → defecto)	{'b': 1}
2. ^a	a	0 (no existe → defecto)	{'b': 1, 'a': 1}
3. ^a	n	0	{'b': 1, 'a': 1, 'n': 1}
4. ^a	a	1 (ya existía)	{'b': 1, 'a': 2, 'n': 1}
5. ^a	n	1	{'b': 1, 'a': 2, 'n': 2}
6. ^a	a	2	{'b': 1, 'a': 3, 'n': 2}

Toda la magia está en `.get(letra, 0)` : si la letra es nueva, finge que valía cero y arranca el contador; si ya estaba, devuelve lo acumulado. **Una sola línea hace de inicialización y de actualización a la vez.**

BUENA PRÁCTICA • EL MISMO PATRÓN, MIL USOS

Cambia «letra» por lo que quieras y tienes: votos por candidato, ventas por producto, visitas por ciudad, errores por tipo, palabras repetidas en un texto. **Cuando escuches «¿cuántas veces aparece cada...?», la respuesta siempre es un diccionario.**

5.6 Las reglas de las claves

REGLA	QUÉ SIGNIFICA EN LA PRÁCTICA
Únicas	No puede haber dos claves iguales. Si escribes la misma dos veces, la última gana y la primera se pierde <i>sin aviso</i> .
Inmutables	Sirven textos, números, booleanos y tuplas . No sirven listas ni otros diccionarios: <code>TypeError: unhashable type: 'list'</code> .
Sensibles a mayúsculas	<code>"Ana" != "ana"</code> . Normaliza con <code>.lower().strip()</code> cuando vengan del usuario.
Con orden de inserción	Desde Python 3.7 se recorren en el orden en que las agregaste. No están ordenadas alfabéticamente: están <i>en orden de llegada</i> .

EN LA VIDA REAL • EL FORMATO DEL INTERNET SE LLAMA JSON

Cuando tu app pide datos a un servidor, lo que llega es prácticamente un diccionario:

```
{"usuario": "ana", "seguidores": 1204, "verificado": false}
```

Ese formato se llama **JSON** y es el idioma en que se hablan todos los programas del mundo. Aprender diccionarios es aprender a leer internet por dentro.

La configuración de cualquier programa —el volumen, el tema oscuro, el idioma— es un diccionario de opciones. Por eso puedes cambiar una sin tocar las demás.

Y tu contraseña, cuando inicias sesión, se verifica buscando tu usuario en una estructura de este tipo: por eso entrar tarda lo mismo si eres el usuario número 3 o el número tres millones.

¿SABÍAS QUÉ?

Buscar un nombre en una lista de un millón de elementos obliga a recorrerla: en el peor caso, un millón de comparaciones. Buscarlo en un diccionario de un millón de entradas cuesta... **prácticamente lo mismo que en uno de diez**. No es exageración didáctica: es una propiedad matemática.

El truco se llama **tabla hash**. Python aplica a la clave una función que la convierte en un número (su *hash*) y usa ese número para calcular *directamente* en qué zona de la memoria debe estar el valor. No busca: **calcula**. Por eso las claves deben ser inmutables: si una clave pudiera cambiar después de guardada, su hash cambiaría y el valor quedaría perdido en una dirección que ya no le corresponde.

Hay un detalle histórico bonito: hasta Python 3.6 los diccionarios *no* conservaban el orden de inserción, y todos los libros advertían que no debías confiar en él. En 2016, una optimización pensada para *ahorrar memoria* tuvo como efecto secundario que el orden se conservara. Gustó tanto que en la versión 3.7 se convirtió en una garantía oficial del lenguaje. Uno de los pocos casos en que un accidente técnico terminó siendo una promesa.

■ LABORATORIO DE LA SESIÓN · «AGENDA DE CONTACTOS»

40 minutos · un archivo `lab5.py` · menú con `while True`

1. Crea `agenda = {}` y un menú de cinco opciones: agregar, consultar, borrar, listar, salir (usa `while True` + `match` + `break` del tomo 2).
2. **Agregar:** pide nombre y teléfono. Normaliza el nombre con `.strip().lower()` antes de guardarlo. Si ya existe, avisa y pregunta si desea reemplazarlo.
3. **Consultar:** usa `.get()` con un mensaje por defecto que cumpla las tres piezas del tomo 2 (qué pasó, qué se esperaba, qué hacer).
4. **Borrar:** protégelo con `if nombre in agenda`. Confirma mostrando el número que se eliminó (pista: `pop`).
5. **Listar:** recorre con `.items()` y muestra los contactos alineados y numerados con `enumerate`. Si la agenda está vacía, dílo con claridad en vez de no imprimir nada.
6. Añade al final del listado: `len(agenda)` contactos registrados.

Entrega: el programa funcionando + tu respuesta escrita a esta pregunta: ¿qué se rompería si guardaras los nombres sin normalizar?

MISIÓN DEL NIVEL 5

«Buscar por nombre»

DIFICULTAD ●●●●○ · 45 MINUTOS · ENTREGA: 5 PROGRAMAS

1. Convierte las dos listas paralelas de 5.1 en un diccionario y muestra el inventario completo con `.items()`, alineado y con el total.
2. Escribe un contador de palabras: pide una frase, sepárala con `frase.split()` y cuenta cuántas veces aparece cada palabra.
3. Traductor mínimo: un diccionario español→inglés de diez palabras. Pide una palabra y responde con `.get()`; si no está, ofrece agregarla al diccionario en ese momento.
4. Encuesta: cuenta votos de una lista como `["azul", "rojo", "azul", "verde", "azul"]` y anuncia el ganador (pista: recorre `.items()` con el patrón campeón del tomo 2).
5. **Rómpelo a propósito:** intenta usar una lista como clave y lee el error `unhashable type`. Después explícalo en dos líneas usando la palabra *immutable*.

NIVEL JEFE: escribe un contador de letras que además muestre **la letra más frecuente** y dibuje un histograma con `"#" * veces`, ordenado de mayor a menor. Necesitarás `sorted()` sobre `.items()` con un `key`. Cuando te funcione, pruébalo con el párrafo entero de esta página: acabas de escribir la herramienta con la que se analizan textos de verdad.

Recompensa desbloqueada: *búsqueda directa*. Tus datos ya tienen nombre propio.

CHECKPOINT · NIVEL 5

1. ¿Qué diferencia hay entre `dic["x"]` y `dic.get("x")` cuando la clave no existe?
2. ¿Por qué una lista no puede ser clave de un diccionario, pero una tupla sí?
3. Explica en una frase qué hace `conteo[letra] = conteo.get(letra, 0) + 1`.

ESTE NIVEL EN 3 IDEAS

- Un diccionario guarda parejas **clave → valor**: se busca por nombre, no por posición.
- `.get()` evita el `KeyError` y permite un valor por defecto.
- `.items()` entrega clave y valor a la vez; con él se cuenta y se resume cualquier cosa.

ROJO

Confundo claves con índices y mi programa se cae con `KeyError`.

ÁMBAR

Creo y leo diccionarios, pero el patrón de contar todavía lo copio.

VERDE

Escribo el contador de memoria y sé cuándo un problema pide diccionario.

Anidar: el mundo real es una lista de fichas

Los datos de verdad no son cinco números sueltos: son cien fichas con seis campos cada una. Aquí aprendes la forma que tienen los datos fuera de los ejercicios.

LO QUE DESBLOQUEARÁS

- 1 Guardar cuadrículas con **listas de listas** y recorrerlas con bucles anidados.
- 2 Modelar registros reales con una **lista de diccionarios**.
- 3 Leer cualquier acceso anidado por complicado que parezca, de fuera hacia dentro.
- 4 Filtrar, promediar y rankear una colección de fichas.

PALABRAS CLAVE

anidar lista de listas
matriz fila columna
lista de diccionarios
registro campo JSON
copia superficial
deepcopy

DEL TOMO 2

Los bucles anidados del nivel 12 existían para esto. Un bucle recorre una fila; dos bucles recorren un plano. Ahora ese plano tiene datos dentro.

6.1 Listas dentro de listas: la matriz

Una lista puede contener cualquier cosa... incluidas otras listas. Eso convierte una fila en una tabla:

matriz.py

```
1 tablero = [
2     [1, 2, 3],      # fila 0
3     [4, 5, 6],      # fila 1
4     [7, 8, 9],      # fila 2
5 ]
6
7 print(tablero[1])    # [4, 5, 6] -> una fila entera
8 print(tablero[1][2]) # 6         -> fila 1, columna 2
9
10 for fila in tablero: # el externo recorre FILAS
11     for valor in fila: # el interno recorre COLUMNAS
12         print(f"{valor:3}", end=" ")
13     print()          # salto de fila: pertenece al externo
```

`tablero[1][2]`

	col 0	col 1	col 2	
fila 0	[1	, 2	, 3	]
fila 1	[4	, 5	, [6]	]
fila 2	[7	, 8	, 9	]

`<-- tablero[1][2]`

Se lee SIEMPRE de izquierda a derecha: primero de qué caja, luego qué casilla.

FIG. 6.1 — DOBLE ÍNDICE. El primer corchete escoge la caja interior; el segundo, la casilla dentro de ella. Con esa regla puedes leer `datos[2][0][1]` sin miedo.

ERROR CLÁSICO · CREAR UNA MATRIZ CON *

`tablero = [[0] * 3] * 3` parece práctico y es una trampa: crea **tres nombres para la misma fila** (es el alias del nivel 2 otra vez). Al cambiar `tablero[0][0]` cambian las tres filas a la vez. La forma correcta: `tablero = [[0] * 3 for _ in range(3)]`, que fabrica una fila nueva por vuelta.

LA ESTRUCTURA DEL MUNDO REAL

Lista de diccionarios. Si solo te llevas una cosa de este tomo para tu primer trabajo, que sea esta. Es la forma en que viajan los datos entre programas.

6.2 Lista de diccionarios: la tabla de verdad

Una matriz sirve para números en cuadrícula. Pero los datos reales tienen **campos con nombre**: un estudiante no es «la posición 2», es un nombre, una edad y unas notas. La combinación que resuelve eso es una lista (muchas fichas) de diccionarios (cada ficha con sus campos).

registros.py

```

1 estudiantes = [
2     {"nombre": "Ana", "edad": 16, "notas": [4.5, 3.8, 5.0]},
3     {"nombre": "Luis", "edad": 17, "notas": [2.5, 3.0, 2.8]},
4     {"nombre": "Sara", "edad": 16, "notas": [4.0, 4.2, 3.9]},
5 ]
6
7 print(estudiantes[0])           # la ficha completa de Ana
8 print(estudiantes[0]["nombre"]) # Ana
9 print(estudiantes[0]["notas"][2]) # 5.0 -> tercera nota de Ana
10 print(len(estudiantes))        # 3 fichas

```

CÓMO LEER UN ACCESO ANIDADO, SIN MIEDO

Lee **de izquierda a derecha**, diciendo en voz alta qué tienes en la mano después de cada corchete:

`estudiantes` → una lista de fichas · `estudiantes[0]` → la primera ficha (un diccionario) · `estudiantes[0]["notas"]` → la lista de notas de esa ficha ·

`estudiantes[0]["notas"][2]` → la tercera nota. **Corchete con número = lista; corchete con texto = diccionario.** Esa pista sola te dice qué tienes en cada paso.

TIP DE DEPURACIÓN · IMPRIME POR CAPAS

Cuando un acceso anidado falle, no adivines: imprime capa por capa hasta que algo no sea lo que esperabas. `print(type(datos), len(datos))`, luego `print(datos[0])`, luego `print(datos[0]["notas"])`. El error siempre está en la primera capa que te sorprende.

6.3 Trabajar con fichas: filtrar, resumir, rankear

Ahora los cuatro patrones del tomo 2 se vuelven poderosos de verdad, porque cada vuelta del bucle te entrega **un registro completo**.

● informe.py

```
1 MINIMA = 3.0
2 mejor = None           # campeón: aún no hay ninguno
3 aprobados = []         # lista acumuladora
4
5 for alumno in estudiantes:
6     notas = alumno["notas"]
7     promedio = sum(notas) / len(notas)
8     alumno["promedio"] = round(promedio, 2)  # campo NUEVO en la ficha
9
10    if promedio >= MINIMA:
11        aprobados.append(alumno["nombre"])
12
13    if mejor is None or promedio > mejor["promedio"]:
14        mejor = alumno
15
16 for alumno in estudiantes:
17     estado = "APROBADO" if alumno["promedio"] >= MINIMA else "EN RIESGO"
18     print(f"{alumno['nombre']:<8} {alumno['promedio']:.2f} {estado}")
19
20 print(f"\nAprobados: {len(aprobados)} de {len(estudiantes)} -> {aprobados}")
21 print(f"Mejor promedio: {mejor['nombre']} con {mejor['promedio']}")
```

SALIDA EN PANTALLA

```
Ana      4.43  APROBADO
Luis     2.77  EN RIESGO
Sara     4.03  APROBADO
```

```
Aprobados: 2 de 3 -> ['Ana', 'Sara']
Mejor promedio: Ana con 4.43
```

DOS DETALLES PROFESIONALES DE ESE CÓDIGO

1. El campeón empieza en `None`. Es la versión correcta del problema que viste en el tomo 2: no inventes un valor inicial (0 o 5.0), empieza vacío y trata la primera vuelta como caso especial con `if mejor is None or ...`. Funciona con cualquier dato, incluidos negativos.
2. Se le agregó un campo a la ficha dentro del bucle. `alumno["promedio"] = ...` enriquece el registro, y como el diccionario es mutable, el cambio queda guardado en la lista. Así trabajan los programas reales: *calculan una vez y guardan el resultado*.

● ranking.py

```
1 # Ordenar fichas por un campo: el remate del nivel
2 ranking = sorted(estudiantes, key=lambda a: a["promedio"], reverse=True)
3
4 for puesto, alumno in enumerate(ranking, start=1):
5     print(f"{puesto}. {alumno['nombre']} — {alumno['promedio']}")
```

SOBRE ESA PALABRA RARA: LAMBDA

Por ahora léela así: «ordena mirando, de cada ficha, su campo promedio». `key=lambda a: a["promedio"]` es la instrucción que le dice a `sorted()` qué mirar de cada elemento. En el tomo 4, cuando aprendas funciones, esta línea dejará de ser una fórmula copiada y pasará a ser evidente. Usarla ahora sin dominarla del todo está permitido: es el andamio, no el edificio.

6.4 Diccionario de listas y diccionario de diccionarios

La misma idea al revés: cuando lo que quieres es **buscar por nombre** y que cada nombre tenga varias cosas asociadas.

• otras_formas.py

```
1 # Diccionario de listas: cada clave guarda varias cosas
2 notas_por_materia = {
3     "Matemáticas": [4.5, 3.8],
4     "Español": [2.5, 3.0, 4.1],
5 }
6 notas_por_materia["Español"].append(3.6)      # agregar a la lista de dentro
7
8 for materia, notas in notas_por_materia.items():
9     print(f"{materia:<14} {len(notas)} notas · promedio {sum(notas)/len(notas):.2f}")
10
11 # Diccionario de diccionarios: fichas buscables por identificador
12 alumnos = {
13     "1001": {"nombre": "Ana", "grupo": "A"},
14     "1002": {"nombre": "Luis", "grupo": "B"},
15 }
16 print(alumnos["1002"]["nombre"])              # Luis, en un solo paso
```

NECESITO...	ESTRUCTURA	EJEMPLO TÍPICO
Una cuadrícula de números	Lista de listas	Tablero de juego, imagen, hoja de cálculo
Muchas fichas que recorro entera	Lista de diccionarios	Estudiantes, productos, publicaciones
Fichas que busco por un identificador	Diccionario de diccionarios	Usuarios por cédula, productos por código
Varias cosas agrupadas por categoría	Diccionario de listas	Notas por materia, tareas por día

PELIGRO REAL • `.COPY()` NO BASTA CUANDO HAY ANIDAMIENTO

El `.copy()` del nivel 2 es una **copia superficial**: copia la caja de afuera, pero las cajas de adentro siguen siendo las mismas.

```
1 respaldo = estudiantes.copy()
2 respaldo[0]["nombre"] = "CAMBIADO"
3 print(estudiantes[0]["nombre"])  # CAMBIADO ← el "respaldo" no
                                  # protegió nada
```

Para una copia real e independiente hay una herramienta hecha: `from copy import deepcopy` y luego `respaldo = deepcopy(estudiantes)`. Guárdalo: el día que un «respaldo» te falle sin explicación, será esto.

EN LA VIDA REAL · ASÍ TE LLEGAN LOS DATOS

Una respuesta de cualquier servicio de internet tiene exactamente esta forma: una lista de fichas con campos con nombre, y dentro de cada ficha, más listas y más fichas. Se llama **JSON** y se lee igual que lo que acabas de escribir.

Tu app de mensajería guarda *chats* (lista) con *mensajes* (lista) que tienen *autor*, *hora* y *texto* (diccionario). Tu plataforma de video guarda *canales* con *videos* con *comentarios*. Todo el software que usas es esta estructura, repetida y ampliada. Cuando la dominas, dejas de ver aplicaciones y empiezas a ver datos.

¿SABÍAS QUÉ?

JSON (JavaScript Object Notation) fue propuesto por Douglas Crockford a principios de los 2000 como una alternativa ligera a los formatos de intercambio de la época, que eran pesados y difíciles de leer. Su enorme éxito se debe a algo casi tonto: **es legible por humanos** y se parece muchísimo a las estructuras que los lenguajes ya tenían.

Tanto se parece que Python trae un módulo para pasar de uno a otro en una línea: `import json` y ya puedes convertir un diccionario en texto guardable, y un texto recibido en un diccionario. Es el tema natural del tomo 4, cuando aprendas archivos — y entonces tus programas dejarán de olvidarlo todo al cerrarse.

■ LABORATORIO DE LA SESIÓN · «BASE DE DATOS DE PELÍCULAS»

45 minutos · `lab6.py` · construye por partes y ejecuta después de cada una

1. Crea una lista con **cinco fichas** de película: título, director, año y una lista de géneros.
2. Muestra el catálogo completo en columnas alineadas, numerado con `enumerate`.
3. Pide un año y muestra solo las películas posteriores (patrón filtrar + lista acumuladora).
4. Calcula y muestra el **año promedio** y la película **más antigua** (patrón campeón empezando en `None`).
5. Cuenta cuántas películas hay **por género**. Ojo: cada ficha tiene *varios* géneros, así que necesitas un bucle dentro de otro y el patrón contador con diccionario del nivel 5.
6. Muestra el catálogo ordenado por año descendente con `sorted(..., key=lambda p: p["año"])`.

Entrega: el programa + un dibujo a mano (sí, en papel) de la estructura de datos que creaste, con sus cajas y flechas. Ese dibujo vale tanto como el código.

MISIÓN DEL NIVEL 6

«Datos con forma»

DIFICULTAD ●●●●○ · 50 MINUTOS · ENTREGA: 4 PROGRAMAS

1. Tablero de tres en raya: crea la matriz 3×3 correctamente (no con `*`), imprímela con bordes y permite marcar una casilla pidiendo fila y columna. Valida el rango.
2. Convierte la lista de tuplas `(nombre, puntaje)` del nivel 4 en una lista de diccionarios y muestra el podio. Compara cuál de las dos versiones se lee mejor y escribe por qué.
3. Inventario de tienda: lista de fichas con nombre, precio y cantidad. Muestra el valor total del inventario y los productos con menos de 5 unidades.
4. **Rómpelo a propósito:** haz `respaldo = datos.copy()`, modifica una ficha del respaldo y comprueba el desastre. Después arréglalo con `deepcopy`.

NIVEL JEFE: agrupa los estudiantes por grupo (A, B, C) en un **diccionario de listas** construido automáticamente a partir de la lista de fichas, y muestra el promedio de cada grupo. Necesitarás la línea `grupos.setdefault(clave, []).append(alumno)` o su equivalente con `if clave not in grupos`. Ese ejercicio —agrupar— es una de las tareas más frecuentes que existen en el trabajo real con datos.

Recompensa desbloqueada: *estructura*. Ya puedes modelar información del mundo real, no solo ejemplos de clase.

CHECKPOINT · NIVEL 6

1. ¿Qué devuelve `datos[2]["notas"][0]` si `datos` es una lista de fichas? Explica cada corchete.
2. ¿Por qué `[[0]*3]*3` es una trampa?
3. ¿Cuándo `.copy()` no alcanza y qué se usa en su lugar?

ESTE NIVEL EN 3 IDEAS

- Una colección puede contener otras: así se representan tablas, fichas y datos reales.
- **Corchete con número = lista; corchete con texto = diccionario.** Con eso lees cualquier acceso anidado.
- La **lista de diccionarios** es la estructura del mundo real, y se llama JSON cuando viaja por internet.

ROJO

Me pierdo entre corchetes y no sé qué tengo en la mano en cada paso.

ÁMBAR

Leo estructuras anidadas, pero recorrerlas con dos bucles todavía me cuesta.

VERDE

Diseño la estructura antes de escribir código y sé justificar por qué elegí esa.

Cazar bugs de colecciones y construir algo tuyo

Los seis errores que vas a cometer, cómo atraparlos en menos de cinco minutos, y el proyecto que junta los siete niveles.

LO QUE DESBLOQUEARÁS

- 1 Identificar los 6 bugs típicos de colecciones **por su síntoma**.
- 2 Usar el *print espía* de estructuras: tipo, tamaño y contenido.
- 3 Elegir la estructura correcta antes de escribir la primera línea.
- 4 Escribir y ampliar un programa completo de unas 100 líneas.

PALABRAS CLAVE

depurar type() len()
 IndexError KeyError
 TypeError alias
 caso borde refactorizar
 modelar

DIAGNÓSTICO PRIMERO

No leas el código buscando el error. Mira el *síntoma*, elige la fila de la tabla y ve directo a la línea sospechosa. Ahorra horas.

7.1 Los seis bugs de colecciones, por su síntoma

LO QUE VES	CASI SEGURO ES...	DÓNDE MIRAR EXACTAMENTE
<code>IndexError: list index out of range</code>	Índice fuera de rango	Pediste la casilla <code>len</code> . El último índice válido es <code>len - 1</code> . Revisa los <code>range()</code> y los <code>+1</code> .
<code>KeyError: 'x'</code>	Clave inexistente	Mayúsculas, espacios o un campo opcional. Cambia el corchete por <code>.get()</code> o valida con <code>in</code> .
<code>AttributeError: 'NoneType'...</code>	Método que devolvió None	Escribiste <code>lista = lista.append(x)</code> o <code>.sort()</code> a la derecha de un igual.
La lista original cambió sola	Alias, no copia	Hay un <code>b = a</code> en vez de <code>b = a.copy()</code> . Con datos anidados, <code>deepcopy</code> .
Se saltó elementos al borrar	Modificar mientras se recorre	Un <code>remove</code> dentro del <code>for</code> que recorre esa misma lista. Recorre <code>.copy()</code> o construye una lista nueva.
El total sale igual al último dato	Acumulador reiniciado	La inicialización quedó <i>dentro</i> del bucle. Debe estar a la misma altura que el <code>for</code> .

ERROR CLÁSICO • EL QUE NO DA ERROR

Los dos últimos de la tabla son los peligrosos: **no producen ningún mensaje**. El programa corre feliz y entrega un número creíble. Por eso la regla de oro de este nivel es probar siempre con datos cuyo resultado *ya conoces*: si sabes que el promedio de `[2, 4]` es 3, y tu programa dice 4, acabas de cazar un bug que nadie te habría avisado.

7.2 El *print espía* de estructuras

Con números bastaba con imprimir el valor. Con colecciones necesitas tres datos: **qué es**, **cuánto mide** y **qué contiene**.

● `espia.py`

```
1 print(f"[DEBUG] tipo={type(datos).__name__} | largo={len(datos)}")
2 print(f"[DEBUG] contenido={datos}")
3
4 # Dentro de un bucle, numera SIEMPRE la vuelta
5 for i, ficha in enumerate(estudiantes):
6     print(f" [DEBUG] {i}: claves={list(ficha.keys())}")
```

CAZA EL BUG • ¿QUÉ TIENE DE MALO?

```
1 notas = [4.5, 2.8, 3.9]
2 total = 0
3
4 for i in range(len(notas) + 1):
5     total = notas[i]
6
7 print(total / len(notas))
```

Hay dos bugs, no uno. El primero grita: `range(len(notas) + 1)` llega hasta el índice 3, que no existe → `IndexError`. El segundo es mudo: dice `total =` en vez de `total +=`, así que aunque arregles el rango, el «total» sería solo la última nota y el promedio saldría 1.3 en lugar de 3.73.

Moraleja: arreglar el error que Python te muestra no significa que el programa esté bien. Después de cada arreglo, **verifica el resultado contra un dato que conozcas**.

PROTOCOLO ANTIPÁNICO PARA COLECCIONES

1. **Reduce el tamaño.** Si falla con 100 fichas, prueba con 2. Un bug con 2 se ve; con 100 se esconde.
2. **Imprime tipo y largo** antes de la línea que falla. La mitad de los bugs se resuelven ahí.
3. **Prueba los bordes:** colección vacía, un solo elemento, todos iguales, con negativos.
4. **Verifica contra un resultado conocido** calculado a mano.
5. **Explícalo en voz alta**, línea por línea. El error suele aparecer solo, a mitad de frase.

7.3 ¿Qué estructura uso? El árbol de decisión

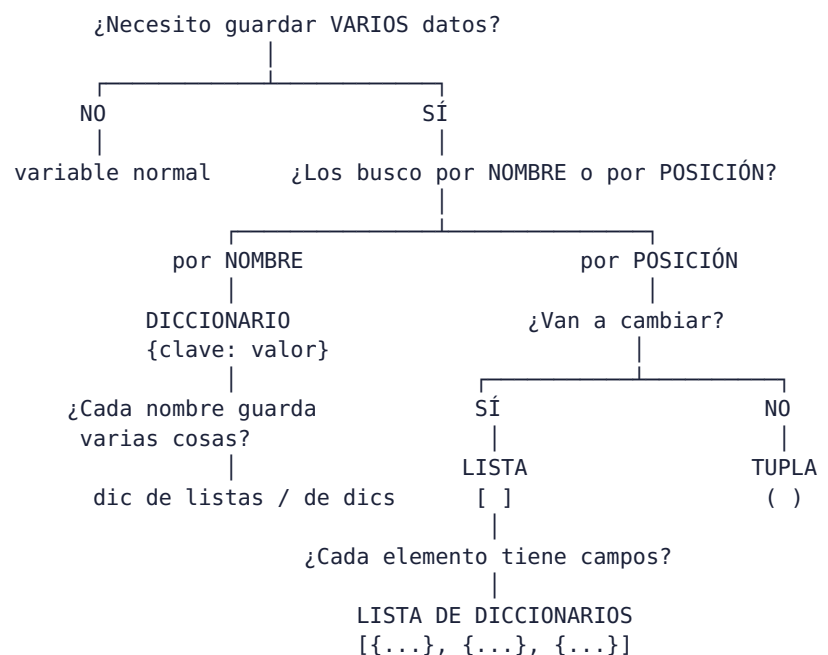


FIG. 7.1 — CÓMO ELEGIR. Dos preguntas resuelven el 95 % de los casos: ¿cómo lo voy a buscar? y ¿va a cambiar? Responderlas antes de escribir código ahorra reescribir el programa entero.

7.4 Proyecto final: Biblioteca Personal

Momento de juntarlo todo. Este programa usa los siete niveles: listas, métodos, ciclos con patrones, tuplas, diccionarios, anidamiento, validación y menú. **Escríbelo por partes, ejecutando después de cada una:** así se construye software de verdad. Si escribes las cuatro de un tirón y falla, tendrás cien líneas donde buscar.

Parte 1 de 4 • El esqueleto y los datos

Ejecuta esta parte tal cual: el menú ya debe funcionar y salir con la opción 6, aunque las demás todavía no hagan nada.

```

1 # =====
2 # BIBLIOTECA PERSONAL · Proyecto del tomo 3
3 # =====
4
5 GENEROS = ("novela", "ciencia", "historia", "comic") # tupla: no cambia
6
7 libros = [                                     # lista de diccionarios
8     {"titulo": "Rayuela", "autor": "Cortázar",
9      "genero": "novela", "paginas": 600, "leido": True},
10    {"titulo": "Cosmos", "autor": "Sagan",
11     "genero": "ciencia", "paginas": 380, "leido": False},
12 ]
13
14 print("=" * 46)
15 print("          BIBLIOTECA PERSONAL")
16 print("=" * 46)
17
18 while True:
19     print("\n1. Ver catálogo      2. Agregar libro")
20     print("3. Buscar              4. Marcar como leído")
21     print("5. Estadísticas          6. Salir")
22     opcion = input("> ").strip()
23
24     match opcion:
25         case "6":
26             print("Cerrando la biblioteca. ¡Hasta la próxima!")
27             break
28         case _:
29             print("Esa opción no existe. Escribe un número del 1 al 6.")

```

Parte 2 de 4 · Ver el catálogo y agregar (validación + append)

Estos dos `case` van antes del `case "6"`.

```

1     case "1":
2         if len(libros) == 0:                # caso borde: vacío
3             print("Todavía no hay libros. Usa la opción 2 para agregar.")
4             continue
5
6         print(f"\n{'#':<3}{'TÍTULO':<22}{'AUTOR':<14}{'GÉNERO':<11}{'EST.':<1}"
7         print("-" * 56)
8         for i, libro in enumerate(libros, start=1):
9             estado = "leído" if libro["leído"] else "pendiente"
10            print(f"{i:<3}{libro['titulo']:<22}{libro['autor']:<14}"
11                  f"{libro['genero']:<11}{estado}")
12
13    case "2":
14        titulo = input("Título: ").strip()
15        while titulo == "":                # validar: no vacío
16            print("El título no puede quedar vacío. Ejemplo: Cosmos")
17            titulo = input("Título: ").strip()
18
19        autor = input("Autor: ").strip()
20
21        print(f"Géneros válidos: {' '.join(GENEROS)}")
22        genero = input("Género: ").strip().lower()
23        while genero not in GENEROS:      # validar contra la tupla
24            print(f"Ese género no está en la lista. Escribe uno de: {' '.join(GENEROS)}")
25            genero = input("Género: ").strip().lower()
26
27        paginas_texto = input("Páginas: ").strip()
28        while not paginas_texto.isdigit(): # validar tipo (tomo 2)
29            print("Necesito solo dígitos, sin letras ni comas. Ejemplo: 380")
30            paginas_texto = input("Páginas: ").strip()
31
32        libros.append({"titulo": titulo, "autor": autor,
33                      "genero": genero, "paginas": int(paginas_texto),
34                      "leído": False})
35        print(f"Lista: «{titulo}» agregado. Ahora tienes {len(libros)} libros.")

```

Parte 3 de 4 • Buscar y marcar (búsqueda con bandera y `break`)

La opción 3 es una búsqueda parcial: recorre todas las fichas y muestra las que coincidan. La opción 4 usa `for ... else`, la joya del tomo 2, para avisar cuando no encontró nada.

```

1     case "3":
2         texto = input("Buscar por título o autor: ").strip().lower()
3         encontrados = []                # lista acumuladora
4
5         for libro in libros:
6             if texto in libro["titulo"].lower() or texto in libro["autor"].lower():
7                 encontrados.append(libro)
8
9         if len(encontrados) == 0:
10            print(f"No hay libros que contengan «{texto}». Prueba con menos letras.")
11        else:
12            for libro in encontrados:
13                print(f"· {libro['titulo']} – {libro['autor']} ({libro['paginas']}
14pág.)")
15
16        case "4":
17            buscado = input("Título exacto del libro leído: ").strip().lower()
18
19            for libro in libros:
20                if libro["titulo"].lower() == buscado:
21                    libro["leído"] = True          # el dic es mutable: se guarda
22                    print(f"«{libro['titulo']}» marcado como leído.")
23                    break                          # ya está: no sigas buscando
24            else:                                  # else del for: NO hubo break
25                print("No encontré ese título. Revisa el catálogo con la opción 1.")

```

● biblioteca.py · parte 4 de 4

```

1     case "5":
2         if len(libros) == 0:
3             print("No hay datos para analizar todavía.")
4             continue
5
6         total_paginas = 0           # acumulador
7         leidos = 0                 # contador
8         mas_largo = None           # campeón
9         por_genero = {}            # contador por categoría
10
11        for libro in libros:
12            total_paginas += libro["paginas"]
13            if libro["leido"]:
14                leidos += 1
15            if mas_largo is None or libro["paginas"] > mas_largo["paginas"]:
16                mas_largo = libro
17            g = libro["genero"]
18            por_genero[g] = por_genero.get(g, 0) + 1
19
20        print(f"\nLibros registrados : {len(libros)}")
21        print(f"Leídos           : {leidos} ({leidos / len(libros) * 100:.0f} %)")
22        print(f"Páginas totales   : {total_paginas:,}")
23        print(f"Promedio de páginas: {total_paginas / len(libros):.0f}")
24        print(f"El más largo      : {mas_largo['titulo']} ({mas_largo['paginas']}
pág.)")
25
26        print("\nPor género:")
27        for genero, cuantos in sorted(por_genero.items()):
28            print(f" {genero:<10} {'█' * cuantos} {cuantos}")

```

SALIDA COMPLETA DE UNA SESIÓN

```
=====
                        BIBLIOTECA PERSONAL
=====

1. Ver catálogo      2. Agregar libro
3. Buscar            4. Marcar como leído
5. Estadísticas      6. Salir
> 2
Título: Sapiens
Autor: Harari
Géneros válidos: novela, ciencia, historia, comic
Género: Historia
Páginas: 460
Listo: «Sapiens» agregado. Ahora tienes 3 libros.

> 1
#  TÍTULO                AUTOR        GÉNERO      EST.
-----
1  Rayuela                Cortázar     novela      leído
2  Cosmos                 Sagan       ciencia     pendiente
3  Sapiens                 Harari       historia    pendiente

> 5
Libros registrados : 3
Leídos             : 1 (33 %)
Páginas totales    : 1,440
Promedio de páginas: 480
El más largo       : Rayuela (600 pág.)

Por género:
ciencia  █ 1
historia █ 1
novela   █ 1

> 6
Cerrando la biblioteca. ¡Hasta la próxima!
```

YA TIENE UN DEFECTO • ENCUÉNTRALO Y ARRÉGLALO

En la opción 2, nada impide agregar **dos veces el mismo título**. Lo dejamos ahí a propósito. Peor aún: si eso ocurre, la opción 4 marcará solo el primero por culpa del `break`, y quien use el programa jurará que «no funciona».

Con lo que aprendiste puedes arreglarlo sin herramientas nuevas: antes del `append`, recorre la lista y comprueba si ya existe un libro con ese título (comparando en minúsculas). Ese arreglo es tu primer trabajo real como programador: **encontrar el punto donde tu propio programa confía demasiado**.

¿SABÍAS QUÉ?

En 1976, Niklaus Wirth —creador del lenguaje Pascal— publicó un libro cuyo título es una de las frases más citadas de la informática: «**Algoritmos + Estructuras de Datos = Programas**».

La idea de fondo es exactamente lo que acabas de vivir en el proyecto: un programa no es solo una secuencia de instrucciones, es *una forma de organizar datos* más *una forma de recorrerlos*. Y de las dos, la que casi siempre decide si el problema es fácil o imposible es la **primera**. Programadores con experiencia lo dicen de otra forma: «*elige bien la estructura y el algoritmo se escribe solo*». Si el nivel jefe del nivel 2 te pareció durísimo y el mismo problema en el nivel 6 te pareció fácil, no fue porque hayas mejorado tanto en dos días: fue porque cambiaste la estructura.

RETO FINAL DEL TOMO

«Hazlo tuyo»

DIFICULTAD ●●●●● · 70 MINUTOS · ENTREGA: TU PROPIO PROGRAMA

1. Escribe la biblioteca completa a mano, parte por parte, ejecutando después de cada una.
2. Arregla el defecto del recuadro ámbar: no permitir títulos duplicados, con un mensaje de las tres piezas.
3. Añade la opción **7 · Pendientes**: muestra solo los libros no leídos, ordenados por páginas de menor a mayor, para poder empezar por el más corto.
4. Añade la opción **8 · Eliminar**: pide el número que aparece en el catálogo, valida que exista y usa `pop()` mostrando qué libro se retiró. Cuidado con el índice: el usuario ve desde 1.
5. Convierte tu biblioteca en otra cosa: inventario de tienda, lista de canciones, agenda de entrenamientos. Cambia solo los campos del diccionario; verás que el **90 % del código sirve igual**. Eso es lo que significa haber elegido bien la estructura.

JEFE FINAL: introduce a propósito, uno por uno, los seis bugs de la tabla 7.1 en tu programa. Para cada uno documenta en tu cuaderno: qué síntoma vio el usuario, cómo lo detectaste y qué línea lo causaba. Ese documento es tu primer manual de depuración de colecciones, y vale más que cualquier resumen que puedas leer.

Recompensa desbloqueada: *arquitecto de datos*. Ya no guardas valores: diseñas la forma en que la información vive dentro de tu programa.

CHECKPOINT · NIVEL FINAL

1. Ves que una lista «cambió sola» después de hacer un respaldo. ¿Cuál es tu primera sospecha y qué línea revisas?
2. ¿Por qué arreglar el error que Python muestra no garantiza que el programa esté bien?
3. En el proyecto, ¿por qué `GENEROS` es una tupla y `libros` una lista?

ESTE NIVEL EN 3 IDEAS

- Los bugs de colecciones son seis y se identifican **por el síntoma** antes que por el código.
- El *print espía* de estructuras muestra **tipo, largo y contenido**; dentro de un bucle, además, la vuelta.
- Elegir bien la estructura al principio ahorra la mitad del programa después.

ROJO

Cuando algo falla, cambio cosas al azar hasta que funcione.

ÁMBAR

Sé usar el print espía, pero todavía no reconozco los bugs por el síntoma.

VERDE

Diagnostico por síntoma, verifico con un dato conocido y arreglo la línea correcta.